



Ho Chi Minh City University of Technology
Faculty of Computer Science and Engineering

Chapter 5: Dynamic Programming and Greedy Algorithms

Algorithms Analysis and Design (C03031)

Instructor: Assoc. Prof. Dr. Vo Thi Ngoc Chau

Email: chauvtn@hcmut.edu.vn



Course Learning Outcomes

1. Able to analyze the complexity of the algorithms (recursive or iterative) and estimate the efficiency of the algorithms.
2. Improve the ability to design algorithms in different areas *by decomposing a computing problem into advanced algorithm design strategies.*
3. Able to discuss NP-completeness.



References

- [1] Cormen, T. H., Leiserson, C. E, and Rivest, R. L., *Introduction to Algorithms*, The MIT Press, 2009.
- [2] Levitin, A., *Introduction to the Design and Analysis of Algorithms*, 3rd Edition, Pearson, 2012.
- [3] Sedgewick, R., *Algorithms in C++*, Addison-Wesley, 1998.
- [4] Weiss, M.A., *Data Structures and Algorithm Analysis in C*, The Benjamin/Cummings Publishing, 1993.



Course Outline

- ❑ Chapter 1. Fundamentals
- ❑ Chapter 2. Divide-and-Conquer Strategy
- ❑ Chapter 3. Decrease-and-Conquer Strategy
- ❑ Chapter 4. Transform-and-Conquer Strategy
- ❑ **Chapter 5. Dynamic Programming and Greedy Strategies**
- ❑ Chapter 6. Backtracking and Branch-and-Bound
- ❑ Chapter 7. NP-completeness
- ❑ Chapter 8. Approximation Algorithms



Chapter 5.

Dynamic Programming and Greedy Strategies

□ 5.1. Dynamic Programming

- 5.1.1. Matrix-chain multiplication
- 5.1.2. Longest common subsequence
- 5.1.3. (0-1) Knapsack problem
- 5.1.4. Transitive closure with Warshall's algorithm
- 5.1.5. The all-pairs shortest path problem with Floyd's algorithm

□ 5.2. Greedy Algorithms

- 5.2.1. Activity-selection problem
- 5.2.2. Fractional knapsack problem
- 5.2.3. Huffman codes
- 5.2.4. Graph coloring



5.1. Dynamic Programming

- ❑ Dynamic programming was invented by Richard Bellman, a US mathematician, in 1950s as a general method for optimizing multistage decision processes.
 - Programming $\approx$ Planning
- ❑ Dynamic programming is a technique for solving problems with *overlapping subproblems*.
 - The subproblems arise from a *recurrence relating a solution to a given problem with solutions to its smaller subproblems of the same type*.
 - Rather than solving overlapping subproblems again and again, *each of the smaller subproblems is solved only once and the result is recorded in a table* from which we can then obtain a solution to the original problem.



5.1. Dynamic Programming

- The development of a dynamic programming algorithm can be divided into a sequence of four steps.
 1. Characterize the structure of an optimal solution.
 2. *Recursively define* the value of an optimal solution.
 - Define the value of an optimal solution recursively in terms of the optimal solutions to subproblems.
 3. Compute the value of an optimal solution in a *bottom-up* fashion.
 4. Construct the final optimal solution from computed information.



5.1. Dynamic Programming

- There are two key elements that an optimization problem must have for dynamic programming to be applicable:
 - (1) optimal substructure
 - (2) overlapping subproblems
- Optimal substructure (principle of optimality):
 - *The optimal solution for the problem contains within it optimal solutions to subproblems.*
- Overlapping subproblems:
 - *When a recursive algorithm revisits the same problem over and over again, we say that the optimization problem has overlapping subproblems.*
 - Dynamic programming algorithms take advantage of overlapping subproblems by solving each subproblem once and then storing the solution in a table where it can be looked up when needed, using constant time per lookup.
 - Dynamic programming algorithms work in a bottom-up fashion.



5.1. Dynamic Programming

- List the sequence of Fibonacci numbers

1, 1, 2, 3, 5, 8, 13, 21, 34, ...

Recurrence relation:

$$F_N = F_{N-1} + F_{N-2}, N \geq 2$$

$$F_0 = F_1 = 1$$

The *iterative* Fibonacci function is designed with *dynamic programming* technique.

F: a table that stores the solutions of the overlapping subproblems.

⇒ *bottom-up* fashion

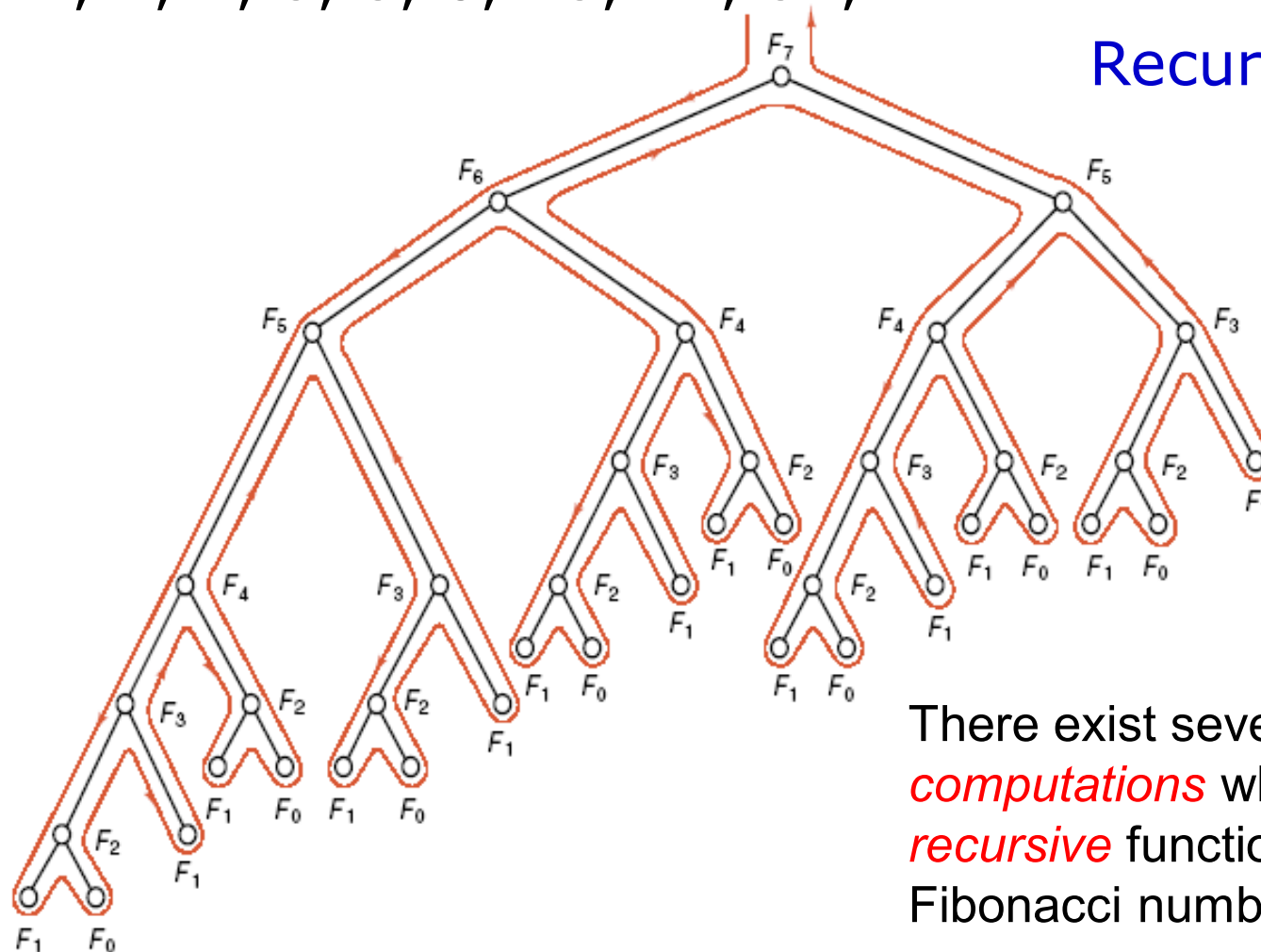
```
function fibonacci_r (N: integer): integer;
begin
  if N <= 1
  then fibonacci := 1
  else fibonacci := fibonacci (N-1) + fibonacci (N-2);
end;

function fibonacci_i (N: integer): integer;
var      i: integer;
          F: array [0..max] of integer; //max>=N
begin
  F[0] := 1; F[1] := 1;
  for i := 2 to N do F[i] := F[i-1] + F[i-2];
  fibonacci := F[N];
end;
```

5.1. Dynamic Programming

- List the sequence of Fibonacci numbers

1, 1, 2, 3, 5, 8, 13, 21, 34, ...



Recursive tree

There exist several *redundant computations* when using the *recursive* function to compute Fibonacci numbers!!!



5.1. Dynamic Programming

- List the sequence of Fibonacci numbers

1, 1, 2, 3, 5, 8, 13, 21, 34, ...

Recurrence relation:

$$F_N = F_{N-1} + F_{N-2}, N \geq 2$$

$$F_0 = F_1 = 1$$

The *recursive* function:

$$O\left(\left(\frac{1 + \sqrt{5}}{2}\right)^N\right)$$

The *iterative* function:

$$O(N)$$

```
function fibonacci_r (N: integer): integer;  
begin  
  if N <= 1  
  then fibonacci := 1  
  else fibonacci := fibonacci (N-1) + fibonacci (N-2);  
end;  
  
function fibonacci_i (N: integer): integer;  
var      i: integer;  
         F: array [0..max] of integer; //max>=N  
begin  
  F[0] := 1; F[1] := 1;  
  for i := 2 to N do F[i] := F[i-1] + F[i-2];  
  fibonacci := F[N];  
end;
```



5.1.1. Matrix-chain multiplication

- Given n matrices $A_1, A_2, \dots, A_n$, we wish to compute their product: $A_1 A_2 \dots A_n$.
- A product of matrices is *fully parenthesized* if it is either a single matrix or the product of two fully-parenthesized matrix products, surrounded by parentheses.
 - $A_1 A_2 A_3 A_4$ can be fully parenthesized in 5 ways:

A_1	10×5
A_2	5×15
A_3	15×25
A_4	25×20

$$(A_1(A_2(A_3A_4)))$$

$$(A_1((A_2A_3)A_4))$$

$$((A_1A_2)(A_3A_4))$$

$$((A_1(A_2A_3))A_4)$$

$$(((A_1A_2)A_3)A_4)$$



5.1.1. Matrix-chain multiplication

- The way we parenthesize a chain of matrices can have a dramatic impact on the *cost* of evaluating the product.

- For example: $A_1 \quad 10 \times 100$

$$A_2 \quad 100 \times 5$$

$$A_3 \quad 5 \times 50$$

- $((A_1 A_2) A_3)$ needs

$$10 \cdot 100 \cdot 5 + 10 \cdot 5 \cdot 50 = 5000 + 2500 = 7500 \text{ scalar multiplications.}$$

- $(A_1 (A_2 A_3))$ needs

$$100 \cdot 5 \cdot 50 + 10 \cdot 100 \cdot 50 = 25000 + 50000 = 75000 \text{ scalar multiplications.}$$

- Computing the product according to the first parenthesization is 10 times faster.



5.1.1. Matrix-chain multiplication

□ The matrix-chain multiplication problem is stated:

“Given a chain $\langle A_1, A_2, \dots, A_n \rangle$ of n matrices, where for $i = 1, 2, \dots, n$, matrix A_i has dimensions $p_{i-1} \times p_i$, fully parenthesize the product $A_1 A_2 \dots A_n$ in such a way that *minimizes* the number of scalar multiplications”.

■ $A_1 A_2 A_3 A_4$ can be fully parenthesized in 5 ways:

A_1	10×5	$(A_1(A_2(A_3A_4)))$
A_2	5×15	$(A_1((A_2A_3)A_4))$
A_3	15×25	$((A_1A_2)(A_3A_4))$
A_4	25×20	$((A_1(A_2A_3))A_4)$
		$((((A_1A_2)A_3)A_4))$

Which way has the smallest number of scalar multiplications?



5.1.1. Matrix-chain multiplication

□ The matrix-chain multiplication problem is stated:

“Given a chain $\langle A_1, A_2, \dots, A_n \rangle$ of n matrices, where for $i = 1, 2, \dots, n$, matrix A_i has dimensions $p_{i-1} \times p_i$, fully parenthesize the product $A_1 A_2 \dots A_n$ in such a way that *minimizes* the number of scalar multiplications”.

- Step 1: Characterize the structure of an optimal solution
- Step 2: *Recursively define* the value of an optimal solution
- Step 3: Compute the value of an optimal solution in a *bottom-up* fashion
- Step 4: Construct the *final optimal solution* from computed information



5.1.1. Matrix-chain multiplication

Step 1: Characterize the structure of an optimal solution

- Let $A_{i..j}$ denote the matrix that is obtained from evaluating the product: $A_i A_{i+1} \dots A_j$.
- An optimal parenthesization of the product $A_1.A_2 \dots A_n$ splits the product between A_k and A_{k+1} for some integer k , $1 \leq k < n$. That is, for some value of k , we first compute the matrices $A_{1..k}$ and $A_{k+1..n}$ and then multiply them together to produce the final product $A_{1..n}$.

$$A_1 A_2 \dots A_n = (A_1 \dots A_k)(A_{k+1} \dots A_n) \text{ for } 1 \leq k < n$$

The cost of this optimal parenthesization =

the cost of computing $A_{1..k}$

+ the cost of computing $A_{k+1..n}$

+ the cost of multiplying them together



5.1.1. Matrix-chain multiplication

Step 2: *Recursively define* the value of an optimal solution

- For the matrix-chain multiplication problem, our subproblems are the problems of determining the minimum cost of a parenthesization of $A_i.A_{i+1} \dots A_j$ for $1 \leq i \leq j \leq n$.
- Let $m[i, j]$ be the minimum number of scalar multiplications needed to compute the matrix $A_{i..j}$. The cost of the cheapest way to compute $A_{1..n}$ is $m[1, n]$.
- Assume that the optimal parenthesization splits the product $A_i A_{i+1} \dots A_j$ between A_k and A_{k+1} , where $i \leq k < j$. Then $m[i, j]$ is equal to the minimum cost of computing the subproducts $A_{i..k}$ and $A_{k+1..j}$, plus the cost of multiplying these two matrices together.

$$m[i, j] = m[i, k] + m[k+1, j] + p_{i-1}p_kp_j$$



5.1.1. Matrix-chain multiplication

Step 2: *Recursively define* the value of an optimal solution

- Thus, our recursive definition for the minimum cost of parenthesizing the product $A_i A_{i+1} \dots A_j$ becomes:

$$m[i, j] = \begin{cases} 0 & \text{if } i = j, \\ \min \{m[i, k] + m[k + 1, j] + p_{i-1}p_k p_j\} & \text{if } i < j, i \leq k < j. \end{cases}$$

- To help us keep track of how to construct an optimal solution, let's define:

$s[i, j]$: the best value of k at which we can split the product $A_i A_{i+1} \dots A_j$ to obtain an optimal parenthesization.



5.1.1. Matrix-chain multiplication

Step 2: *Recursively define* the value of an optimal solution

- The important observation with the principle of optimality for the matrix-chain multiplication problem

“The full parenthesization for the subchain $A_1A_2\dots A_k$ inside the optimal parenthesization for the chain $A_1A_2\dots A_n$ must also be an optimal parenthesization.”

- So the optimal solution for the matrix-chain multiplication problem contains within it optimal solutions to subproblems.



5.1.1. Matrix-chain multiplication

Step 3: Compute the value of an optimal solution in a *bottom-up* fashion

- ❑ Instead of computing the solution to recurrence formula by a recursive algorithm, we *compute the optimal cost by using a bottom-up approach*.
- ❑ Assume that A_i has dimensions $p_{i-1} \times p_i$ for $i = 1..n$.
- ❑ The input is a sequence $\langle p_0, p_1, \dots, p_n \rangle$.
- ❑ The procedure uses table $m[1..n, 1..n]$ for storing the cost $m[i, j]$ and table $s[1..n, 1..n]$ that records the best value k for the optimal cost of $m[i, j]$.
- ❑ For $i < j$, the part of m above the main diagonal is used.
- ❑ MATRIX-CHAIN-ORDER returns two tables m and s .



5.1.1. Matrix-chain multiplication

```
procedure MATRIX-CHAIN-ORDER(p, m, s);  
begin  
  n := length[p] - 1;  
  for i := 1 to n do m[i, i] := 0;  
  for l := 2 to n do /* l: length of the chain = the number of matrices */  
    for i:= 1 to n - l + 1 do  
      begin  
        j := i + l - 1;  
        m[i, j] := ∞; /* initialization */  
        for k := i to j-1 do /*find k for an optimal parenthesization*/  
          begin  
            q := m[i, k] + m[k + 1, j] + pi-1pkpj;  
            if q < m[i, j] then  
              begin m[i, j] := q; s[i, j] := k end  
            end  
          end  
        end  
      end  
    end  
  end
```



5.1.1. Matrix-chain multiplication

Step 4: Construct the *final optimal solution* from computed information

- We use table $s[1..n, 1..n]$ to determine the best way to multiply the matrices. Each entry $s[i, j]$ records the value of k such that the optimal parenthesization of $A_i A_{i+1} \dots A_j$ splits the product between A_k and A_{k+1} .
- Given the matrices $A = \langle A_1, A_2, \dots, A_n \rangle$, table s computed by MATRIX-CHAIN-ORDER and the indices i and j , the following recursive procedure MATRIX-CHAIN-MULTIPLY computes the matrix chain product $A_{i..j}$. The procedure returns the result in parameter AIJ .
- With the call: MATRIX-CHAIN-MULTIPLY($A, s, 1, n, A1N$), the procedure returns the matrix chain product with the parameter $A1N$.



5.1.1. Matrix-chain multiplication

```
procedure MATRIX-CHAIN-MULTIPLY(A, s, i, j, AIJ);  
begin  
  if  $j > i$  then  
    begin  
      MATRIX-CHAIN-MULTIPLY(A, s, i, s[i, j], X);  
      MATRIX-CHAIN-MULTIPLY(A, s, s[i, j]+1, j, Y);  
      MATRIX-MULTIPLY(X, Y, AIJ);  
    end  
  else  
    assign  $A_i$  to AIJ;  
  end;
```



5.1.1. Matrix-chain multiplication

Given the following matrices:

$$A_1 \quad 30 \times 35$$

$$A_2 \quad 35 \times 15$$

$$A_3 \quad 15 \times 5$$

$$A_4 \quad 5 \times 10$$

$$A_5 \quad 10 \times 20$$

$$A_6 \quad 20 \times 25$$

$$p_0 = 30; p_1 = 35; p_2 = 15; p_3 = 5; p_4 = 10; p_5 = 20; p_6 = 25$$

What are tables m and s ? What is the most cost-effective way for *parenthesization* to perform this matrix-chain multiplication problem?



5.1.1. Matrix-chain multiplication

Table m

		i					
		1	2	3	4	5	6
j	6	15125	10500	5375	3500	5000	0
	5	11875	7125	2500	1000	0	
	4	9375	4375	750	0		
	3	7875	2625	0			
	2	15750	0				
	1	0					

Table s

	1	2	3	4	5	6
6	3	3	3	5	5	-
5	3	3	3	4	-	
4	3	3	3	-		
3	1	2	-			
2	1	-				
1	-					

Result:

$(A1 (A2 A3)) ((A4 A5) A6)$



5.1.1. Matrix-chain multiplication

Table m

$l = 2, 1 k:$

Compute:

$k=1: m[1, 2]$

$k=2: m[2, 3]$

$k=3: m[3, 4]$

$k=4: m[4, 5]$

$k=5: m[5, 6]$

$l = 2,$
 $1 k$

j	i					
	1	2	3	4	5	6
6	15125	10500	5375	3500	5000	0
5	11875	7125	2500	1000	0	
4	9375	4375	750	0		
3	7875	2625	0			
2	15750	0				
1	0					

Table s

$l = 2,$
 $1 k$

	1	2	3	4	5	6
6	3	3	3	5	5	-
5	3	3	3	4	-	
4	3	3	3	-		
3	1	2	-			
2	1	-				
1	-					



5.1.1. Matrix-chain multiplication

Table m

$l = 3, 2 k$:

Compute:

$k=1, 2: m[1, 3]$

$k=2, 3: m[2, 4]$

$k=3, 4: m[3, 5]$

$k=4, 5: m[4, 6]$

$l = 3,$
 $2 k$

j	i					
	1	2	3	4	5	6
6	15125	10500	5375	3500	5000	0
5	11875	7125	2500	1000	0	
4	9375	4375	750	0		
3	7875	2625	0			
2	15750	0				
1	0					

Table s

$l = 3,$
 $2 k$

	1	2	3	4	5	6
6	3	3	3	5	5	-
5	3	3	3	4	-	
4	3	3	3	-		
3	1	2	-			
2	1	-				
1	-					



5.1.1. Matrix-chain multiplication

Table m

$l = 4, 3 k$:

Compute:

$k=1, 2, 3: m[1, 4]$

$k=2, 3, 4: m[2, 5]$

$k=3, 4, 5: m[3, 6]$

$l = 4,$
 $3 k$

		i					
		1	2	3	4	5	6
j	6	15125	10500	5375	3500	5000	0
	5	11875	7125	2500	1000	0	
	4	9375	4375	750	0		
	3	7875	2625	0			
	2	15750	0				
	1	0					

Table s

$l = 4,$
 $3 k$

		1	2	3	4	5	6
	6	3	3	3	5	5	-
	5	3	3	3	4	-	
	4	3	3	3	-		
	3	1	2	-			
	2	1	-				
	1	-					



5.1.1. Matrix-chain multiplication

Table m

		i					
		1	2	3	4	5	6
j	6	15125	10500	5375	3500	5000	0
	5	11875	7125	2500	1000	0	
	4	9375	4375	750	0		
	3	7875	2625	0			
	2	15750	0				
	1	0					

$l = 5, 4 k:$

Compute:

$k=1, 2, 3, 4: m[1, 5]$

$k=2, 3, 4, 5: m[2, 6]$

$l = 5,$
 $4 k$

Table s

		1	2	3	4	5	6
j	6	3	3	3	5	5	-
	5	3	3	3	4	-	
	4	3	3	3	-		
	3	1	2	-			
	2	1	-				
	1	-					

$l = 5,$
 $4 k$



5.1.1. Matrix-chain multiplication

Table m

$l = 6,$
 $5 \leq k$

	<i>i</i>					
	1	2	3	4	5	6
6	15125	10500	5375	3500	5000	0
5	11875	7125	2500	1000	0	
4	9375	4375	750	0		
3	7875	2625	0			
2	15750	0				
1	0					

$l = 6, 5 \leq k$:

Compute:

$k=1, 2, 3, 4, 5: m[1, 6]$

Table s

$l = 6,$
 $5 \leq k$

	1	2	3	4	5	6
6	3	3	3	5	5	-
5	3	3	3	4	-	
4	3	3	3	-		
3	1	2	-			
2	1	-				
1	-					30



5.1.1. Matrix-chain multiplication

□ $p_0 = 30; p_1 = 35; p_2 = 15; p_3 = 5; p_4 = 10; p_5 = 20; p_6 = 25$

□ Compute tables m and s :

■ $m[1, 2] = p_0 p_1 p_2 = 30.35.15 = \mathbf{15750}$
 $\rightarrow k=1 \rightarrow s[1, 2] = 1$

■ $m[2, 3] = p_1 p_2 p_3 = 35.15.5 = \mathbf{2625}$
 $\rightarrow k=2 \rightarrow s[2, 3] = 2$

■ $m[3, 4] = p_2 p_3 p_4 = 15.5.10 = \mathbf{750}$
 $\rightarrow k=3 \rightarrow s[3, 4] = 3$

■ $m[4, 5] = p_3 p_4 p_5 = 5.10.20 = \mathbf{1000}$
 $\rightarrow k=4 \rightarrow s[4, 5] = 4$

■ $m[5, 6] = p_4 p_5 p_6 = 10.20.25 = \mathbf{5000}$
 $\rightarrow k=5 \rightarrow s[5, 6] = 5$



5.1.1. Matrix-chain multiplication

□ $p_0 = 30; p_1 = 35; p_2 = 15; p_3 = 5; p_4 = 10; p_5 = 20; p_6 = 25$

□ Compute tables m and s :

$$m[1,3] = \min \begin{cases} m[1,1] + m[2,3] + p_0 p_1 p_3 = 0 + 2625 + 30 \cdot 35 \cdot 5 = \mathbf{7875} \\ m[1,2] + m[3,3] + p_0 p_2 p_3 = 15750 + 0 + 30 \cdot 15 \cdot 5 = 18000 \end{cases}$$

$$\rightarrow k = 1 \rightarrow s[1,3] = 1$$

$$m[2,4] = \min \begin{cases} m[2,2] + m[3,4] + p_1 p_2 p_4 = 0 + 750 + 35 \cdot 15 \cdot 10 = 6000 \\ m[2,3] + m[4,4] + p_1 p_3 p_4 = 2625 + 0 + 35 \cdot 5 \cdot 10 = \mathbf{4375} \end{cases}$$

$$\rightarrow k = 3 \rightarrow s[2,4] = 3$$

$$m[3,5] = \min \begin{cases} m[3,3] + m[4,5] + p_2 p_3 p_5 = 0 + 1000 + 15 \cdot 5 \cdot 20 = \mathbf{2500} \\ m[3,4] + m[5,5] + p_2 p_4 p_5 = 750 + 0 + 15 \cdot 10 \cdot 20 = 5250 \end{cases}$$

$$\rightarrow k = 3 \rightarrow s[3,5] = 3$$

$$m[4,6] = \min \begin{cases} m[4,4] + m[5,6] + p_3 p_4 p_6 = 0 + 5000 + 5 \cdot 10 \cdot 25 = 6250 \\ m[4,5] + m[6,6] + p_3 p_5 p_6 = 1000 + 0 + 5 \cdot 20 \cdot 25 = \mathbf{3500} \end{cases}$$

$$\rightarrow k = 5 \rightarrow s[4,6] = 5$$



5.1.1. Matrix-chain multiplication

□ $p_0 = 30; p_1 = 35; p_2 = 15; p_3 = 5; p_4 = 10; p_5 = 20; p_6 = 25$

□ Compute tables m and s :

$$m[1,4] = \min \begin{cases} m[1,1] + m[2,4] + p_0 p_1 p_4 = 0 + 4375 + 30 \cdot 35 \cdot 10 = 14875 \\ m[1,2] + m[3,4] + p_0 p_2 p_4 = 15750 + 750 + 30 \cdot 15 \cdot 10 = 21000 \\ m[1,3] + m[4,4] + p_0 p_3 p_4 = 7875 + 0 + 30 \cdot 5 \cdot 10 = \mathbf{9375} \end{cases}$$

$$\rightarrow k = 3 \rightarrow s[1,4] = 3$$

$$m[2,5] = \min \begin{cases} m[2,2] + m[3,5] + p_1 p_2 p_5 = 0 + 2500 + 35 \cdot 15 \cdot 20 = 13000 \\ m[2,3] + m[4,5] + p_1 p_3 p_5 = 2625 + 1000 + 35 \cdot 5 \cdot 20 = \mathbf{7125} \\ m[2,4] + m[5,5] + p_1 p_4 p_5 = 4375 + 0 + 35 \cdot 10 \cdot 20 = 11375 \end{cases}$$

$$\rightarrow k = 3 \rightarrow s[2,5] = 3$$

$$m[3,6] = \min \begin{cases} m[3,3] + m[4,6] + p_2 p_3 p_6 = 0 + 3500 + 15 \cdot 5 \cdot 25 = \mathbf{5375} \\ m[3,4] + m[5,6] + p_2 p_4 p_6 = 750 + 5000 + 15 \cdot 10 \cdot 25 = 9500 \\ m[3,5] + m[6,6] + p_2 p_5 p_6 = 2500 + 0 + 15 \cdot 20 \cdot 25 = 10000 \end{cases}$$

$$\rightarrow k = 3 \rightarrow s[3,6] = 3$$



5.1.1. Matrix-chain multiplication

- $p_0 = 30; p_1 = 35; p_2 = 15; p_3 = 5; p_4 = 10; p_5 = 20; p_6 = 25$
- Compute tables m and s :

$$m[1,5] = \min \begin{cases} m[1,1] + m[2,5] + p_0 p_1 p_5 = 0 + 7125 + 30 \cdot 35 \cdot 20 = 28125 \\ m[1,2] + m[3,5] + p_0 p_2 p_5 = 15750 + 2500 + 30 \cdot 15 \cdot 20 = 27250 \\ m[1,3] + m[4,5] + p_0 p_3 p_5 = 7875 + 1000 + 30 \cdot 5 \cdot 20 = \mathbf{11875} \\ m[1,4] + m[5,5] + p_0 p_4 p_5 = 9375 + 0 + 30 \cdot 10 \cdot 20 = 15375 \end{cases}$$

$$\rightarrow k = 3 \rightarrow s[1, 5] = 3$$

$$m[2,6] = \min \begin{cases} m[2,2] + m[3,6] + p_1 p_2 p_6 = 0 + 5375 + 35 \cdot 15 \cdot 25 = 18500 \\ m[2,3] + m[4,6] + p_1 p_3 p_6 = 2625 + 3500 + 35 \cdot 5 \cdot 25 = \mathbf{10500} \\ m[2,4] + m[5,6] + p_1 p_4 p_6 = 4375 + 5000 + 35 \cdot 10 \cdot 25 = 18125 \\ m[2,5] + m[6,6] + p_1 p_5 p_6 = 7125 + 0 + 35 \cdot 20 \cdot 25 = 24625 \end{cases}$$

$$\rightarrow k = 3 \rightarrow s[2, 6] = 3$$



5.1.1. Matrix-chain multiplication

- $p_0 = 30; p_1 = 35; p_2 = 15; p_3 = 5; p_4 = 10; p_5 = 20; p_6 = 25$
- Compute tables m and s :

$$m[1,6] = \min \begin{cases} m[1,1] + m[2,6] + p_0 p_1 p_6 = 0 + 10500 + 30.35.25 = 36750 \\ m[1,2] + m[3,6] + p_0 p_2 p_6 = 15750 + 5375 + 30.15.25 = 32375 \\ m[1,3] + m[4,6] + p_0 p_3 p_6 = 7875 + 3500 + 30.5.25 = \mathbf{15125} \\ m[1,4] + m[5,6] + p_0 p_4 p_6 = 9375 + 5000 + 30.10.25 = 21875 \\ m[1,5] + m[6,6] + p_0 p_5 p_6 = 11875 + 0 + 30.20.25 = 26875 \end{cases}$$

$$\rightarrow k = 3 \rightarrow s[1, 6] = 3$$



5.1.1. Matrix-chain multiplication

Table m

j	i					
	1	2	3	4	5	6
6	15125	10500	5375	3500	5000	0
5	11875	7125	2500	1000	0	
4	9375	4375	750	0		
3	7875	2625	0			
2	15750	0				
1	0					

Table s

	1	2	3	4	5	6
6	3	3	3	5	5	-
5	3	3	3	4	-	
4	3	3	3	-		
3	1	2	-			
2	1	-				
1	-					

How do we derive *the optimal parenthesization* from table s ?

Result:

$(A1 (A2 A3)) ((A4 A5) A6)$



5.1.1. Matrix-chain multiplication

Table m

		i					
		1	2	3	4	5	6
j	6	15125	10500	5375	3500	5000	0
	5	11875	7125	2500	1000	0	
	4	9375	4375	750	0		
	3	7875	2625	0			
	2	15750	0				
	1	0					

Table s

	1	2	3	4	5	6
6	3	3	3	5	5	-
5	3	3	3	4	-	
4	3	3	3	-		
3	1	2	-			
2	1	-				
1	-					

How do we derive the optimal parenthesization from table s ?

Result:

(A1 A2 A3) (A4 A5 A6)



5.1.1. Matrix-chain multiplication

Table m

		i					
		1	2	3	4	5	6
j	6	15125	10500	5375	3500	5000	0
	5	11875	7125	2500	1000	0	
	4	9375	4375	750	0		
	3	7875	2625	0			
	2	15750	0				
	1	0					

Table s

	1	2	3	4	5	6
6	3	3	3	5	5	-
5	3	3	3	4	-	
4	3	3	3	-		
3	1	2	-			
2	1	-				
1	-					

How do we derive the optimal parenthesization from table s ?

Result:

(A1 (A2 A3)) (A4 A5 A6)



5.1.1. Matrix-chain multiplication

Table m

		i					
		1	2	3	4	5	6
j	6	15125	10500	5375	3500	5000	0
	5	11875	7125	2500	1000	0	
	4	9375	4375	750	0		
	3	7875	2625	0			
	2	15750	0				
	1	0					

Table s

	1	2	3	4	5	6
6	3	3	3	5	5	-
5	3	3	3	4	-	
4	3	3	3	-		
3	1	2	-			
2	1	-				
1	-					

How do we derive the
optimal parenthesization
from table s ?

Result:

$(A1 (A2 A3)) ((A4 A5) A6)$



5.1.1. Matrix-chain multiplication

MATRIX-CHAIN-MULTIPLY(A, s, 1, 6, A1n) // $s[1,6] = 3 \rightarrow (A1\ A2\ A3)(A4\ A5\ A6)$

MATRIX-CHAIN-MULTIPLY(A, s, 1, 3, X) // $s[1,3] = 1 \rightarrow (A1\ (A2\ A3))$

MATRIX-CHAIN-MULTIPLY(A, s, 1, 1, XX)

A1 $\rightarrow$ XX

MATRIX-CHAIN-MULTIPLY(A, s, 2, 3, YY) // $s[2,3] = 2$

MATRIX-CHAIN-MULTIPLY(A, s, 2, 2, XXX)

A2 $\rightarrow$ XXX

MATRIX-CHAIN-MULTIPLY(A, s, 3, 3, YYY)

A3 $\rightarrow$ YYY

MATRIX-MULTIPLY(XXX, YYY, YY) // $YY = A2 * A3$

MATRIX-MULTIPLY(XX, YY, X) // $X = A1 * (A2 * A3)$

MATRIX-CHAIN-MULTIPLY(A, s, 4, 6, Y) // $s[4,6] = 5 \rightarrow ((A4\ A5)\ A6)$

MATRIX-CHAIN-MULTIPLY(A, s, 4, 5, XX) // $s[4,5] = 4$

MATRIX-CHAIN-MULTIPLY(A, s, 4, 4, XXX)

A4 $\rightarrow$ XXX

MATRIX-CHAIN-MULTIPLY(A, s, 5, 5, YYY)

A5 $\rightarrow$ YYY

MATRIX-MULTIPLY(XXX, YYY, XX) // $XX = A4 * A5$

MATRIX-CHAIN-MULTIPLY(A, s, 6, 6, YY)

A6 $\rightarrow$ YY

MATRIX-MULTIPLY(XX, YY, Y) // $Y = (A4 * A5) * A6$

MATRIX-MULTIPLY(X, Y, A1n) // $A1n = (A1 * (A2 * A3))((A4 * A5) * A6)$



5.1.1. Matrix-chain multiplication

- Time complexity of the dynamic programming algorithm for matrix-chain multiplication problem to fully parenthesize the product $A_1A_2...A_n$ in such a way that *minimizes* the number of scalar multiplications for the product
 - Abstract operation: *multiplication*
 - Best, Average, Worst cases:

$$C_n = O(n^3)$$



5.1.1. Matrix-chain multiplication

```
procedure MATRIX-CHAIN-ORDER(p, m, s);
begin
  n := length[p] - 1;
  for i := 1 to n do m[i, i] := 0;
  for l := 2 to n do /* l: length of the chain = the number of matrices */
    for i := 1 to n - l + 1 do
      begin
        j := i + l - 1;
        m[i, j] := ∞; /* initialization */
        for k := i to j-1 do /* find k for an optimal parenthesization */
          begin
            q := m[i, k] + m[k + 1, j] + pi-1pkpj;
            if q < m[i, j] then
              begin m[i, j] := q; s[i, j] := k end
            end
          end
        end
      end
    end
  end
```

Abstract operation:
multiplication



5.1.1. Matrix-chain multiplication

- Time complexity of the dynamic programming algorithm for matrix-chain multiplication problem to fully parenthesize the product $A_1A_2...A_n$ in such a way that *minimizes* the number of scalar multiplications for the product
 - Abstract operation: *multiplication*
 - Best, Average, Worst cases:

$$C_n = O(n^3)$$

Iterative algorithm: $C_n = \sum_{l=2..n} \sum_{i=1..n-l+1} \sum_{k=i..i+l-2} 2$



5.1.1. Matrix-chain multiplication

- Time complexity of the dynamic programming algorithm
 - Abstract operation: *multiplication*
 - Best, Average, Worst cases:

$$C_n = O(n^3)$$

$$\begin{aligned} C_n &= \sum_{l=2}^n \sum_{i=1}^{n-l+1} \sum_{k=i}^{i+l-2} 2 = \sum_{l=2}^n \sum_{i=1}^{n-l+1} 2(i + l - 2 - i + 1) = \sum_{l=2}^n \sum_{i=1}^{n-l+1} 2(l - 1) \\ C_n &= \sum_{l=2}^n 2(l - 1)(n - l + 1 - 1 + 1) = \sum_{l=2}^n 2(l - 1)(n - l + 1) \\ C_n &= 2n \frac{n(n - 1)}{2} - 2 \frac{(n - 1)n(2n - 1)}{6} = \frac{n(n - 1)(3n - 2n + 1)}{3} \end{aligned}$$



5.1.1. Matrix-chain multiplication

- Time complexity of the dynamic programming algorithm
 - Abstract operation: *multiplication*
 - Best, Average, Worst cases:

$$C_n = O(n^3)$$

$$C_n = \sum_{l=2}^n \sum_{i=1}^{n-l+1} \sum_{k=i}^{i+l-2} 2 = \sum_{l=2}^n \sum_{i=1}^{n-l+1} 2(i + l - 2 - i + 1) = \sum_{l=2}^n \sum_{i=1}^{n-l+1} 2(l - 1)$$

$$C_n = 2n \frac{n(n-1)}{2} - 2 \frac{(n-1)n(2n-1)}{6} = \frac{n(n-1)(3n-2n+1)}{3}$$

$$C_n = \frac{n(n-1)(n+1)}{3} = O(n^3)$$



5.1.1. Matrix-chain multiplication

□ Your turn:

- 5.1.1.1. Given the following matrices, how different is $(A_1A_2)A_3$ from $A_1(A_2A_3)$ in terms of cost?

$$A_1: 3 \times 1000, \quad A_2: 1000 \times 3, \quad A_3: 3 \times 1000$$

- Given the following matrices, what are tables m and s ? What is the most cost-effective way for *parenthesization* to perform the matrix-chain multiplication problem?

5.1.1.2.

A_1	10×5
A_2	5×15
A_3	15×25
A_4	25×20

5.1.1.3.

A_1	8×3
A_2	3×2
A_3	2×19
A_4	19×18

5.1.1.4.

A_1	5×10
A_2	10×4
A_3	4×6
A_4	6×10
A_5	10×2



5.1.2. Longest common subsequence

- Given $X = \langle x_1, x_2, \dots, x_m \rangle$. X is called a sequence. For example, a sequence is $X = \langle A, B, C, B, D, A, B \rangle$.
- Given $Z = \langle B, C, D, B \rangle$. Z is called a subsequence of X with the corresponding index sequence $\langle 2, 3, 5, 7 \rangle$.
- A *subsequence* of a sequence is just the given sequence with some elements left out.
- Given two sequences X and Y , Z is a *common subsequence* of X and Y if Z is a subsequence of both X and Y .
- In the longest-common-subsequence problem, given two sequences $X = \langle x_1, x_2, \dots, x_m \rangle$ and $Y = \langle y_1, y_2, \dots, y_n \rangle$, we wish to find a *maximum length common subsequence* (*LCS*) of X and Y .



5.1.2. Longest common subsequence

- In the longest-common-subsequence problem, given two sequences $X = \langle x_1, x_2, \dots, x_m \rangle$ and $Y = \langle y_1, y_2, \dots, y_n \rangle$, we wish to find a *maximum length common subsequence* (*LCS*) of X and Y .
- Given two sequences X and Y as follows:
$$X = \langle A, B, C, B, D, A, B \rangle$$
$$Y = \langle B, D, C, A, B, A \rangle$$
 - $\langle B, D, A, B \rangle$, $\langle B, C, B, A \rangle$, and $\langle B, C, A, B \rangle$ are LCS's of X and Y .
 - How to find them *efficiently*?



5.1.2. Longest common subsequence

- In the longest-common-subsequence problem, given two sequences $X = \langle x_1, x_2, \dots, x_m \rangle$ and $Y = \langle y_1, y_2, \dots, y_n \rangle$, we wish to find a *maximum length common subsequence* (*LCS*) of X and Y .
 - Step 1: Characterize the structure of an optimal solution
 - Step 2: *Recursively define* the value of an optimal solution
 - Step 3: Compute the value of an optimal solution in a *bottom-up* fashion
 - Step 4: Construct the *final optimal solution* from computed information



5.1.2. Longest common subsequence

Step 1: Characterize the structure of an optimal solution

- Given a sequence $X = \langle x_1, x_2, \dots, x_m \rangle$, we define the *i*-th prefix of X , for $i = 0, 1, \dots, m$, is $X_i = \langle x_1, x_2, \dots, x_i \rangle$.
- Let $X = \langle x_1, x_2, \dots, x_m \rangle$ and $Y = \langle y_1, y_2, \dots, y_n \rangle$ be sequences, $Z = \langle z_1, z_2, \dots, z_k \rangle$ be any LCS of X and Y .
 1. If $x_m = y_n$ then $z_k = x_m = y_n$ and Z_{k-1} is LCS of X_{m-1} and Y_{n-1} .
 2. If $x_m \neq y_n$ then $z_k \neq x_m$ implies that Z is LCS of X_{m-1} and Y .
 3. If $x_m \neq y_n$ then $z_k \neq y_n$ implies that Z is LCS of X and Y_{n-1} .
- To find an LCS of X and Y , we need to find the LCS's of X_{m-1} and Y_{n-1} , X and Y_{n-1} , X_{m-1} and Y . Each of these subproblems has the subsubproblem of finding the LCS.



5.1.2. Longest common subsequence

Step 2: *Recursively define* the value of an optimal solution

- Let's define $c[i, j]$ to be the length of LCS of the subsequences X_i and Y_j . If either $i = 0$ or $j = 0$, one of the sequences has length 0, so the LCS has length 0. The optimal substructure of the LCS problem gives the recursive formula:

$$c[i, j] = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0 \\ c[i-1, j-1] + 1 & \text{if } i, j > 0 \text{ and } x_i = y_j \\ \max(c[i, j-1], c[i-1, j]) & \text{if } i, j > 0 \text{ and } x_i \neq y_j \end{cases}$$



5.1.2. Longest common subsequence

Step 3: Compute the value of an optimal solution in a *bottom-up* fashion

- ❑ Based on the recursive formula, we could write a recursive algorithm to compute the length of an LCS of two sequences. However, we compute the solutions in a *bottom-up* fashion.
- ❑ Procedure LCS-LENGTH takes two sequences $X = \langle x_1, x_2, \dots, x_m \rangle$ và $Y = \langle y_1, y_2, \dots, y_n \rangle$ as inputs.
- ❑ It stores the $c[i, j]$ values in table $c[0..m, 0..n]$. It also maintains table $b[1..m, 1..n]$ to simplify construction of an optimal solution.

5.1.2. Longest common subsequence

procedure LCS-LENGTH(X, Y)

begin

$m := \text{length}[X]; n := \text{length}[Y];$

for $i := 1$ **to** m **do** $c[i, 0] := 0;$

for $j := 1$ **to** n **do** $c[0, j] := 0;$

for $i := 1$ **to** m **do**

for $j := 1$ **to** n **do**

if $x_i = y_j$ **then**

begin $c[i, j] := c[i-1, j-1] + 1; b[i, j] := \nwarrow$ **end**

else if $c[i-1, j] \geq c[i, j-1]$ **then**

begin $c[i, j] := c[i-1, j]; b[i, j] := \uparrow$ **end**

else

begin $c[i, j] := c[i, j-1]; b[i, j] := \leftarrow$ **end**

end;

	$j-1$	j
$i-1$	$[i-1, j-1]$	$[i-1, j]$
i	$[i, j-1]$	$[i, j]$





5.1.2. Longest common subsequence

Step 4: Construct the *final optimal solution* from computed information

The table b is used to construct an LCS of

$X = \langle x_1, x_2, \dots, x_m \rangle$ and
 $Y = \langle y_1, y_2, \dots, y_n \rangle$

The recursive procedure prints out an LCS of X and Y . The initial invocation is:

PRINT-LCS(b, X, m, n).

```
procedure PRINT-LCS( $b, X, i, j$ )
begin
  if  $i \neq 0$  and  $j \neq 0$  then
    if  $b[i, j] = \text{"↖"}$  then
      begin
        PRINT-LCS( $b, X, i - 1, j - 1$ );
        print  $x_i$ 
      end
    else if  $b[i, j] = \text{"↑"}$  then
      PRINT-LCS( $b, X, i - 1, j$ )
    else PRINT-LCS( $b, X, i, j - 1$ )
  end;
```



5.1.2. Longest common subsequence

- Given two sequences X and Y as follows.
 - Compute tables c and b and then, find the maximum length of their LCS.
 - List one LCS from table b .

$X = \langle A, B, C, B, D, A, B \rangle$

$Y = \langle B, D, C, A, B, A \rangle$



5.1.2. Longest common subsequence

	y_j	B	D	C	A	B	A
x_i	0	0	0	0	0	0	0
A	0	0 $\uparrow$	0 $\uparrow$	0 $\uparrow$	1 $\nwarrow$	1 $\leftarrow$	1 $\nwarrow$
B	0	1 $\nwarrow$	1 $\leftarrow$	1 $\leftarrow$	1 $\uparrow$	2 $\nwarrow$	2 $\leftarrow$
C	0	1 $\uparrow$	1 $\uparrow$	2 $\nwarrow$	2 $\leftarrow$	2 $\uparrow$	2 $\uparrow$
B	0	1 $\nwarrow$	1 $\uparrow$	2 $\uparrow$	2 $\uparrow$	3 $\nwarrow$	3 $\leftarrow$
D	0	1 $\uparrow$	2 $\nwarrow$	2 $\uparrow$	2 $\uparrow$	3 $\uparrow$	3 $\uparrow$
A	0	1 $\uparrow$	2 $\uparrow$	2 $\uparrow$	3 $\nwarrow$	3 $\uparrow$	4 $\nwarrow$
B	0	1 $\nwarrow$	2 $\uparrow$	2 $\uparrow$	3 $\uparrow$	4 $\nwarrow$	4 $\uparrow$

Tables

$c[0..7, 0..6]$

$b[1..7, 1..6]$



5.1.2. Longest common subsequence

	y_j	B	D	C	A	B	A
x_i	0	0	0	0	0	0	0
A	0	0 ↑	0 ↑	0 ↑	1 ↖	1 ←	1 ↖
B	0	1 ↖	1 ←	1 ←	1 ↑	2 ↖	2 ←
C	0	1 ↑	1 ↑	2 ↖	2 ←	2 ↑	2 ↑
B	0	1 ↖	1 ↑	2 ↑	2 ↑	3 ↖	3 ←
D	0	1 ↑	2 ↖	2 ↑	2 ↑	3 ↑	3 ↑
A	0	1 ↑	2 ↑	2 ↑	3 ↖	3 ↑	4 ↖
B	0	1 ↖	2 ↑	2 ↑	3 ↑	4 ↖	4 ↑

Tables

$c[0..7, 0..6]$

$b[1..7, 1..6]$

$LCS =$

<B, C, B, A>



5.1.2. Longest common subsequence

	y_j	B	D	C	A	B	A
x_i	0	0	0	0	0	0	0
A	0						
B	0						
C	0						
B	0						
D	0						
A	0						
B	0						

Tables

$c[0..7, 0..6]$

$b[1..7, 1..6]$



5.1.2. Longest common subsequence

	y_j	B	D	C	A	B	A
x_i	0	0	0	0	0	0	0
A	0	0 ↑					
B	0						
C	0						
B	0						
D	0						
A	0						
B	0						

Tables

$c[0..7, 0..6]$

$b[1..7, 1..6]$



5.1.2. Longest common subsequence

	y_j	B	D	C	A	B	A
x_i	0	0	0	0	0	0	0
A	0	0 ↑	0 ↑				
B	0						
C	0						
B	0						
D	0						
A	0						
B	0						

Tables

$c[0..7, 0..6]$

$b[1..7, 1..6]$



5.1.2. Longest common subsequence

	y_j	B	D	C	A	B	A
x_i	0	0	0	0	0	0	0
A	0	0 ↑	0 ↑	0 ↑			
B	0						
C	0						
B	0						
D	0						
A	0						
B	0						

Tables

$c[0..7, 0..6]$

$b[1..7, 1..6]$



5.1.2. Longest common subsequence

	y_j	B	D	C	A	B	A
x_i	0	0	0	0	0	0	0
A	0	0 ↑	0 ↑	0 ↑	1 ↖		
B	0						
C	0						
B	0						
D	0						
A	0						
B	0						

Tables

$c[0..7, 0..6]$

$b[1..7, 1..6]$



5.1.2. Longest common subsequence

	y_j	B	D	C	A	B	A
x_i	0	0	0	0	0	0	0
A	0	0 ↑	0 ↑	0 ↑	1 ↖	1 ←	
B	0						
C	0						
B	0						
D	0						
A	0						
B	0						

Tables

$c[0..7, 0..6]$

$b[1..7, 1..6]$



5.1.2. Longest common subsequence

	y_j	B	D	C	A	B	A
x_i	0	0	0	0	0	0	0
A	0	0 $\uparrow$	0 $\uparrow$	0 $\uparrow$	1 $\nwarrow$	1 $\leftarrow$	1 $\nwarrow$
B	0						
C	0						
B	0						
D	0						
A	0						
B	0						

Tables

$c[0..7, 0..6]$

$b[1..7, 1..6]$



5.1.2. Longest common subsequence

	y_j	B	D	C	A	B	A
x_i	0	0	0	0	0	0	0
A	0	0 ↑	0 ↑	0 ↑	1 ↖	1 ←	1 ↖
B	0	1 ↖					
C	0						
B	0						
D	0						
A	0						
B	0						

Tables

$c[0..7, 0..6]$

$b[1..7, 1..6]$



5.1.2. Longest common subsequence

	y_j	B	D	C	A	B	A
x_i	0	0	0	0	0	0	0
A	0	0 $\uparrow$	0 $\uparrow$	0 $\uparrow$	1 $\nwarrow$	1 $\leftarrow$	1 $\nwarrow$
B	0	1 $\nwarrow$	1 $\leftarrow$				
C	0						
B	0						
D	0						
A	0						
B	0						

Tables

$c[0..7, 0..6]$

$b[1..7, 1..6]$



5.1.2. Longest common subsequence

	y_j	B	D	C	A	B	A
x_i	0	0	0	0	0	0	0
A	0	0 $\uparrow$	0 $\uparrow$	0 $\uparrow$	1 $\nwarrow$	1 $\leftarrow$	1 $\nwarrow$
B	0	1 $\nwarrow$	1 $\leftarrow$	1 $\leftarrow$			
C	0						
B	0						
D	0						
A	0						
B	0						

Tables

$c[0..7, 0..6]$

$b[1..7, 1..6]$



5.1.2. Longest common subsequence

	y_j	B	D	C	A	B	A
x_i	0	0	0	0	0	0	0
A	0	0 $\uparrow$	0 $\uparrow$	0 $\uparrow$	1 $\nwarrow$	1 $\leftarrow$	1 $\nwarrow$
B	0	1 $\nwarrow$	1 $\leftarrow$	1 $\leftarrow$	1 $\uparrow$		
C	0						
B	0						
D	0						
A	0						
B	0						

Tables

$c[0..7, 0..6]$

$b[1..7, 1..6]$



5.1.2. Longest common subsequence

	y_j	B	D	C	A	B	A
x_i	0	0	0	0	0	0	0
A	0	0 $\uparrow$	0 $\uparrow$	0 $\uparrow$	1 $\nwarrow$	1 $\leftarrow$	1 $\nwarrow$
B	0	1 $\nwarrow$	1 $\leftarrow$	1 $\leftarrow$	1 $\uparrow$	2 $\nwarrow$	
C	0						
B	0						
D	0						
A	0						
B	0						

Tables

$c[0..7, 0..6]$

$b[1..7, 1..6]$



5.1.2. Longest common subsequence

	y_j	B	D	C	A	B	A
x_i	0	0	0	0	0	0	0
A	0	0 $\uparrow$	0 $\uparrow$	0 $\uparrow$	1 $\nwarrow$	1 $\leftarrow$	1 $\nwarrow$
B	0	1 $\nwarrow$	1 $\leftarrow$	1 $\leftarrow$	1 $\uparrow$	2 $\nwarrow$	2 $\leftarrow$
C	0	1 $\uparrow$	1 $\uparrow$	2 $\nwarrow$	2 $\leftarrow$	2 $\uparrow$	2 $\uparrow$
B	0	1 $\nwarrow$	1 $\uparrow$	2 $\uparrow$	2 $\uparrow$	3 $\nwarrow$	3 $\leftarrow$
D	0	1 $\uparrow$	2 $\nwarrow$	2 $\uparrow$	2 $\uparrow$	3 $\uparrow$	3 $\uparrow$
A	0	1 $\uparrow$	2 $\uparrow$	2 $\uparrow$	3 $\nwarrow$	3 $\uparrow$	4 $\nwarrow$
B	0	1 $\nwarrow$	2 $\uparrow$	2 $\uparrow$	3 $\uparrow$	4 $\nwarrow$	4 $\uparrow$

Tables

$c[0..7, 0..6]$

$b[1..7, 1..6]$



5.1.2. Longest common subsequence

	y_j	B	D	C	A	B	A
x_i	0	0	0	0	0	0	0
A	0	0 $\uparrow$	0 $\uparrow$	0 $\uparrow$	1 $\nwarrow$	1 $\leftarrow$	1 $\nwarrow$
B	0	1 $\nwarrow$	1 $\leftarrow$	1 $\leftarrow$	1 $\uparrow$	2 $\nwarrow$	2 $\leftarrow$
C	0	1 $\uparrow$	1 $\uparrow$	2 $\nwarrow$	2 $\leftarrow$	2 $\uparrow$	2 $\uparrow$
B	0	1 $\nwarrow$	1 $\uparrow$	2 $\uparrow$	2 $\uparrow$	3 $\nwarrow$	3 $\leftarrow$
D	0	1 $\uparrow$	2 $\nwarrow$	2 $\uparrow$	2 $\uparrow$	3 $\uparrow$	3 $\uparrow$
A	0	1 $\uparrow$	2 $\uparrow$	2 $\uparrow$	3 $\nwarrow$	3 $\uparrow$	4 $\nwarrow$
B	0	1 $\nwarrow$	2 $\uparrow$	2 $\uparrow$	3 $\uparrow$	4 $\nwarrow$	4 $\uparrow$

Tables

$c[0..7, 0..6]$

$b[1..7, 1..6]$

$LCS =$

$\langle ?, ?, ?, ? \rangle$



5.1.2. Longest common subsequence

	y_j	B	D	C	A	B	A
x_i	0	0	0	0	0	0	0
A	0	0 $\uparrow$	0 $\uparrow$	0 $\uparrow$	1 $\nwarrow$	1 $\leftarrow$	1 $\nwarrow$
B	0	1 $\nwarrow$	1 $\leftarrow$	1 $\leftarrow$	1 $\uparrow$	2 $\nwarrow$	2 $\leftarrow$
C	0	1 $\uparrow$	1 $\uparrow$	2 $\nwarrow$	2 $\leftarrow$	2 $\uparrow$	2 $\uparrow$
B	0	1 $\nwarrow$	1 $\uparrow$	2 $\uparrow$	2 $\uparrow$	3 $\nwarrow$	3 $\leftarrow$
D	0	1 $\uparrow$	2 $\nwarrow$	2 $\uparrow$	2 $\uparrow$	3 $\uparrow$	3 $\uparrow$
A	0	1 $\uparrow$	2 $\uparrow$	2 $\uparrow$	3 $\nwarrow$	3 $\uparrow$	4 $\nwarrow$
B	0	1 $\nwarrow$	2 $\uparrow$	2 $\uparrow$	3 $\uparrow$	4 $\nwarrow$	4 $\uparrow$

Tables

$c[0..7, 0..6]$

$b[1..7, 1..6]$

$LCS =$

$\langle ?, ?, ?, ? \rangle$



5.1.2. Longest common subsequence

	y_j	B	D	C	A	B	A
x_i	0	0	0	0	0	0	0
A	0	0 $\uparrow$	0 $\uparrow$	0 $\uparrow$	1 $\nwarrow$	1 $\leftarrow$	1 $\nwarrow$
B	0	1 $\nwarrow$	1 $\leftarrow$	1 $\leftarrow$	1 $\uparrow$	2 $\nwarrow$	2 $\leftarrow$
C	0	1 $\uparrow$	1 $\uparrow$	2 $\nwarrow$	2 $\leftarrow$	2 $\uparrow$	2 $\uparrow$
B	0	1 $\nwarrow$	1 $\uparrow$	2 $\uparrow$	2 $\uparrow$	3 $\nwarrow$	3 $\leftarrow$
D	0	1 $\uparrow$	2 $\nwarrow$	2 $\uparrow$	2 $\uparrow$	3 $\uparrow$	3 $\uparrow$
A	0	1 $\uparrow$	2 $\uparrow$	2 $\uparrow$	3 $\nwarrow$	3 $\uparrow$	4 $\nwarrow$
B	0	1 $\nwarrow$	2 $\uparrow$	2 $\uparrow$	3 $\uparrow$	4 $\nwarrow$	4 $\uparrow$

Tables

$c[0..7, 0..6]$

$b[1..7, 1..6]$

$LCS =$

$\langle ?, ?, ?, ? \rangle$



5.1.2. Longest common subsequence

	y_j	B	D	C	A	B	A
x_i	0	0	0	0	0	0	0
A	0	0 $\uparrow$	0 $\uparrow$	0 $\uparrow$	1 $\nwarrow$	1 $\leftarrow$	1 $\nwarrow$
B	0	1 $\nwarrow$	1 $\leftarrow$	1 $\leftarrow$	1 $\uparrow$	2 $\nwarrow$	2 $\leftarrow$
C	0	1 $\uparrow$	1 $\uparrow$	2 $\nwarrow$	2 $\leftarrow$	2 $\uparrow$	2 $\uparrow$
B	0	1 $\nwarrow$	1 $\uparrow$	2 $\uparrow$	2 $\uparrow$	3 $\nwarrow$	3 $\leftarrow$
D	0	1 $\uparrow$	2 $\nwarrow$	2 $\uparrow$	2 $\uparrow$	3 $\uparrow$	3 $\uparrow$
A	0	1 $\uparrow$	2 $\uparrow$	2 $\uparrow$	3 $\nwarrow$	3 $\uparrow$	4 $\nwarrow$
B	0	1 $\nwarrow$	2 $\uparrow$	2 $\uparrow$	3 $\uparrow$	4 $\nwarrow$	4 $\uparrow$

Tables

$c[0..7, 0..6]$

$b[1..7, 1..6]$

$LCS =$

$\langle ?, ?, ?, ? \rangle$



5.1.2. Longest common subsequence

	y_j	B	D	C	A	B	A
x_i	0	0	0	0	0	0	0
A	0	0 $\uparrow$	0 $\uparrow$	0 $\uparrow$	1 $\nwarrow$	1 $\leftarrow$	1 $\nwarrow$
B	0	1 $\nwarrow$	1 $\leftarrow$	1 $\leftarrow$	1 $\uparrow$	2 $\nwarrow$	2 $\leftarrow$
C	0	1 $\uparrow$	1 $\uparrow$	2 $\nwarrow$	2 $\leftarrow$	2 $\uparrow$	2 $\uparrow$
B	0	1 $\nwarrow$	1 $\uparrow$	2 $\uparrow$	2 $\uparrow$	3 $\nwarrow$	3 $\leftarrow$
D	0	1 $\uparrow$	2 $\nwarrow$	2 $\uparrow$	2 $\uparrow$	3 $\uparrow$	3 $\uparrow$
A	0	1 $\uparrow$	2 $\uparrow$	2 $\uparrow$	3 $\nwarrow$	3 $\uparrow$	4 $\nwarrow$
B	0	1 $\nwarrow$	2 $\uparrow$	2 $\uparrow$	3 $\uparrow$	4 $\nwarrow$	4 $\uparrow$

Tables

$c[0..7, 0..6]$

$b[1..7, 1..6]$

$LCS =$

$\langle ?, ?, ?, ? \rangle$



5.1.2. Longest common subsequence

	y_j	B	D	C	A	B	A
x_i	0	0	0	0	0	0	0
A	0	0 $\uparrow$	0 $\uparrow$	0 $\uparrow$	1 $\nwarrow$	1 $\leftarrow$	1 $\nwarrow$
B	0	1 $\nwarrow$	1 $\leftarrow$	1 $\leftarrow$	1 $\uparrow$	2 $\nwarrow$	2 $\leftarrow$
C	0	1 $\uparrow$	1 $\uparrow$	2 $\nwarrow$	2 $\leftarrow$	2 $\uparrow$	2 $\uparrow$
B	0	1 $\nwarrow$	1 $\uparrow$	2 $\uparrow$	2 $\uparrow$	3 $\nwarrow$	3 $\leftarrow$
D	0	1 $\uparrow$	2 $\nwarrow$	2 $\uparrow$	2 $\uparrow$	3 $\uparrow$	3 $\uparrow$
A	0	1 $\uparrow$	2 $\uparrow$	2 $\uparrow$	3 $\nwarrow$	3 $\uparrow$	4 $\nwarrow$
B	0	1 $\nwarrow$	2 $\uparrow$	2 $\uparrow$	3 $\uparrow$	4 $\nwarrow$	4 $\uparrow$

Tables

$c[0..7, 0..6]$

$b[1..7, 1..6]$

$LCS =$

$\langle ?, ?, ?, ? \rangle$



5.1.2. Longest common subsequence

	y_j	B	D	C	A	B	A
x_i	0	0	0	0	0	0	0
A	0	0 $\uparrow$	0 $\uparrow$	0 $\uparrow$	1 $\nwarrow$	1 $\leftarrow$	1 $\nwarrow$
B	0	1 $\nwarrow$	1 $\leftarrow$	1 $\leftarrow$	1 $\uparrow$	2 $\nwarrow$	2 $\leftarrow$
C	0	1 $\uparrow$	1 $\uparrow$	2 $\nwarrow$	2 $\leftarrow$	2 $\uparrow$	2 $\uparrow$
B	0	1 $\nwarrow$	1 $\uparrow$	2 $\uparrow$	2 $\uparrow$	3 $\nwarrow$	3 $\leftarrow$
D	0	1 $\uparrow$	2 $\nwarrow$	2 $\uparrow$	2 $\uparrow$	3 $\uparrow$	3 $\uparrow$
A	0	1 $\uparrow$	2 $\uparrow$	2 $\uparrow$	3 $\nwarrow$	3 $\uparrow$	4 $\nwarrow$
B	0	1 $\nwarrow$	2 $\uparrow$	2 $\uparrow$	3 $\uparrow$	4 $\nwarrow$	4 $\uparrow$

Tables

$c[0..7, 0..6]$

$b[1..7, 1..6]$

$LCS =$

$\langle ?, ?, ?, ? \rangle$



5.1.2. Longest common subsequence

	y_j	B	D	C	A	B	A
x_i	0	0	0	0	0	0	0
A	0	0 $\uparrow$	0 $\uparrow$	0 $\uparrow$	1 $\nwarrow$	1 $\leftarrow$	1 $\nwarrow$
B	0	1 $\nwarrow$	1 $\leftarrow$	1 $\leftarrow$	1 $\uparrow$	2 $\nwarrow$	2 $\leftarrow$
C	0	1 $\uparrow$	1 $\uparrow$	2 $\nwarrow$	2 $\leftarrow$	2 $\uparrow$	2 $\uparrow$
B	0	1 $\nwarrow$	1 $\uparrow$	2 $\uparrow$	2 $\uparrow$	3 $\nwarrow$	3 $\leftarrow$
D	0	1 $\uparrow$	2 $\nwarrow$	2 $\uparrow$	2 $\uparrow$	3 $\uparrow$	3 $\uparrow$
A	0	1 $\uparrow$	2 $\uparrow$	2 $\uparrow$	3 $\nwarrow$	3 $\uparrow$	4 $\nwarrow$
B	0	1 $\nwarrow$	2 $\uparrow$	2 $\uparrow$	3 $\uparrow$	4 $\nwarrow$	4 $\uparrow$

Tables

$c[0..7, 0..6]$

$b[1..7, 1..6]$

$LCS =$

$\langle ?, ?, ?, ? \rangle$



5.1.2. Longest common subsequence

	y_j	B	D	C	A	B	A
x_i	0	0	0	0	0	0	0
A	0	0 $\uparrow$	0 $\uparrow$	0 $\uparrow$	1 $\nwarrow$	1 $\leftarrow$	1 $\nwarrow$
B	0	1 $\nwarrow$	1 $\leftarrow$	1 $\leftarrow$	1 $\uparrow$	2 $\nwarrow$	2 $\leftarrow$
C	0	1 $\uparrow$	1 $\uparrow$	2 $\nwarrow$	2 $\leftarrow$	2 $\uparrow$	2 $\uparrow$
B	0	1 $\nwarrow$	1 $\uparrow$	2 $\uparrow$	2 $\uparrow$	3 $\nwarrow$	3 $\leftarrow$
D	0	1 $\uparrow$	2 $\nwarrow$	2 $\uparrow$	2 $\uparrow$	3 $\uparrow$	3 $\uparrow$
A	0	1 $\uparrow$	2 $\uparrow$	2 $\uparrow$	3 $\nwarrow$	3 $\uparrow$	4 $\nwarrow$
B	0	1 $\nwarrow$	2 $\uparrow$	2 $\uparrow$	3 $\uparrow$	4 $\nwarrow$	4 $\uparrow$

Tables

$c[0..7, 0..6]$

$b[1..7, 1..6]$

$LCS =$

$\langle ?, ?, ?, ? \rangle$



5.1.2. Longest common subsequence

	y_j	B	D	C	A	B	A
x_i	0	0	0	0	0	0	0
A	0	0 ↑	0 ↑	0 ↑	1 ↖	1 ←	1 ↖
B	0	1 ↖	1 ←	1 ←	1 ↑	2 ↖	2 ←
C	0	1 ↑	1 ↑	2 ↖	2 ←	2 ↑	2 ↑
B	0	1 ↖	1 ↑	2 ↑	2 ↑	3 ↖	3 ←
D	0	1 ↑	2 ↖	2 ↑	2 ↑	3 ↑	3 ↑
A	0	1 ↑	2 ↑	2 ↑	3 ↖	3 ↑	4 ↖
B	0	1 ↖	2 ↑	2 ↑	3 ↑	4 ↖	4 ↑

Tables

$c[0..7, 0..6]$

$b[1..7, 1..6]$

$LCS =$

$\langle ?, ?, ?, ? \rangle$



5.1.2. Longest common subsequence

	y_j	B	D	C	A	B	A
x_i	0	0	0	0	0	0	0
A	0	0 ↑	0 ↑	0 ↑	1 ↖	1 ←	1 ↖
B	0	1 ↖	1 ←	1 ←	1 ↑	2 ↖	2 ←
C	0	1 ↑	1 ↑	2 ↖	2 ←	2 ↑	2 ↑
B	0	1 ↖	1 ↑	2 ↑	2 ↑	3 ↖	3 ←
D	0	1 ↑	2 ↖	2 ↑	2 ↑	3 ↑	3 ↑
A	0	1 ↑	2 ↑	2 ↑	3 ↖	3 ↑	4 ↖
B	0	1 ↖	2 ↑	2 ↑	3 ↑	4 ↖	4 ↑

Tables

$c[0..7, 0..6]$

$b[1..7, 1..6]$

$LCS =$

<B, ?, ?, ?>



5.1.2. Longest common subsequence

	y_j	B	D	C	A	B	A
x_i	0	0	0	0	0	0	0
A	0	0 ↑	0 ↑	0 ↑	1 ↖	1 ←	1 ↖
B	0	1 ↖	1 ←	1 ←	1 ↑	2 ↖	2 ←
C	0	1 ↑	1 ↑	2 ↖	2 ←	2 ↑	2 ↑
B	0	1 ↖	1 ↑	2 ↑	2 ↑	3 ↖	3 ←
D	0	1 ↑	2 ↖	2 ↑	2 ↑	3 ↑	3 ↑
A	0	1 ↑	2 ↑	2 ↑	3 ↖	3 ↑	4 ↖
B	0	1 ↖	2 ↑	2 ↑	3 ↑	4 ↖	4 ↑

Tables

$c[0..7, 0..6]$

$b[1..7, 1..6]$

$LCS =$

<B, C, ?, ?>



5.1.2. Longest common subsequence

	y_j	B	D	C	A	B	A
x_i	0	0	0	0	0	0	0
A	0	0 ↑	0 ↑	0 ↑	1 ↖	1 ←	1 ↖
B	0	1 ↖	1 ←	1 ←	1 ↑	2 ↖	2 ←
C	0	1 ↑	1 ↑	2 ↖	2 ←	2 ↑	2 ↑
B	0	1 ↖	1 ↑	2 ↑	2 ↑	3 ↖	3 ←
D	0	1 ↑	2 ↖	2 ↑	2 ↑	3 ↑	3 ↑
A	0	1 ↑	2 ↑	2 ↑	3 ↖	3 ↑	4 ↖
B	0	1 ↖	2 ↑	2 ↑	3 ↑	4 ↖	4 ↑

Tables

$c[0..7, 0..6]$

$b[1..7, 1..6]$

$LCS =$

<B, C, B, ?>



5.1.2. Longest common subsequence

	y_j	B	D	C	A	B	A
x_i	0	0	0	0	0	0	0
A	0	0 ↑	0 ↑	0 ↑	1 ↖	1 ←	1 ↖
B	0	1 ↖	1 ←	1 ←	1 ↑	2 ↖	2 ←
C	0	1 ↑	1 ↑	2 ↖	2 ←	2 ↑	2 ↑
B	0	1 ↖	1 ↑	2 ↑	2 ↑	3 ↖	3 ←
D	0	1 ↑	2 ↖	2 ↑	2 ↑	3 ↑	3 ↑
A	0	1 ↑	2 ↑	2 ↑	3 ↖	3 ↑	4 ↖
B	0	1 ↖	2 ↑	2 ↑	3 ↑	4 ↖	4 ↑

Tables

$c[0..7, 0..6]$

$b[1..7, 1..6]$

$LCS =$

<B, C, B, A>



5.1.2. Longest common subsequence

- Time complexity for computing the length of an LCS of two sequences $X = \langle x_1, x_2, \dots, x_m \rangle$ and $Y = \langle y_1, y_2, \dots, y_n \rangle$
 - Abstract operation: *assignment*
 - Best, Average, Worst cases:

$$C_{LCS} = 2mn + m + n = O(mn)$$

5.1.2. Longest common subsequence

procedure LCS-LENGTH(X, Y)

begin

$m := \text{length}[X]; n := \text{length}[Y];$

for $i := 1$ **to** m **do** $c[i, 0] := 0;$

for $j := 1$ **to** n **do** $c[0, j] := 0;$

for $i := 1$ **to** m **do**

for $j := 1$ **to** n **do**

if $x_i = y_j$ **then**

begin $c[i, j] := c[i-1, j-1] + 1;$ $b[i, j] := \text{“}\nwarrow\text{”}$ **end**

else if $c[i-1, j] \geq c[i, j-1]$ **then**

begin $c[i, j] := c[i-1, j];$ $b[i, j] := \text{“}\uparrow\text{”}$ **end**

else

begin $c[i, j] := c[i, j-1];$ $b[i, j] := \text{“}\leftarrow\text{”}$ **end**

end;

	j-1	j
i-1	<u>$[i-1, j-1]$</u>	<u>$[i-1, j]$</u>
i	<u>$[i, j-1]$</u>	<u>$[i, j]$</u>





5.1.2. Longest common subsequence

Your turn:

- Given two sequences X and Y as follows.
 - Compute tables c and b and then, find the maximum length of their LCS.
 - List one LCS from table b .

5.1.2.1.

$X = \langle C, B, A, E, D \rangle$

$Y = \langle A, E, C, A, D, B \rangle$

5.1.2.2.

$X = \langle x, z, y, z, z, y, x \rangle$

$Y = \langle z, x, y, y, z, x, z \rangle$

5.1.2.3.

$X = \langle A, B, A, Z, E, C \rangle$

$Y = \langle B, A, B, A, E \rangle$

5.1.2.4.

$X = \langle x, y, x, x, z, x, y, z, x, y \rangle$

$Y = \langle z, x, z, y, y, z, x, x, y, x, x, z \rangle$



5.1.2. Longest common subsequence

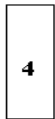
Your turn:

- ❑ 5.1.2.5. Analyze the time complexity of PRINT-LCS algorithm to derive one LCS of X and Y using table b.
- ❑ 5.1.2.6. Modify PRINT-LCS algorithm to derive all the LCSs of X and Y using table b. Reapply your modified PRINT-LCS algorithm to the previous questions 5.1.2.1-5.1.2.4. How about the time complexity of your modified PRINT-LCS algorithm.



5.1.3. (0-1) Knapsack problem

- “A thief robbing a store finds it filled with N types of items of varying size and value, but has only a small knapsack of capacity M to use to carry the goods. *The knapsack problem* is to find the combination of items which the thief should choose for his knapsack in order to maximize the total value of all the items he takes.”

					
name	A	B	C	D	E
size	3	4	7	8	9
value	4	5	10	11	13

Knapsack capacity: $M = 17$

Assume that each type has *the unlimited number* of items.

What is the *combination of items* that *maximizes* the total value of the knapsack?



5.1.3. (0-1) Knapsack problem

- “A thief robbing a store finds it filled with N types of items of varying size and value, but has only a small knapsack of capacity M to use to carry the goods. *The knapsack problem* is to find the combination of items which the thief should choose for his knapsack in order to maximize the total value of all the items he takes.”
- Solved using dynamic programming as follows:
 - Step 1: Characterize the structure of an optimal solution
 - Step 2: *Recursively define* the value of an optimal solution
 - Step 3: Compute the value of an optimal solution in a *bottom-up* fashion
 - Step 4: Construct the *final optimal solution* from computed information



5.1.3. (0-1) Knapsack problem

Step 1: Characterize the structure of an optimal solution

- Let $cost[0..M]$ be an array that stores the highest value achieved with a knapsack of varying capacity $i = 0..M$.
- Let $best[0..M]$ be an array that stores the last item that was added into the knapsack to achieve the maximum value $cost[0..M]$.
- For capacity i and item j , if adding item j makes $cost[i]$ the highest value, $cost[i]$ and $best[i]$ are:

$$cost[i] = cost[i - size[j]] + value[j]$$

$$best[i] = j$$



5.1.3. (0-1) Knapsack problem

Step 2: *Recursively define* the value of an optimal solution

$$cost[i] = \begin{cases} \max\{cost[i], cost[i - size[j]] + value[j]\} & \text{if } i - size[j] \geq 0 \\ \text{unchanged} & \text{if } i - size[j] < 0 \end{cases}$$

- ❑ This formula allows as many items j as possible to be added into the knapsack.
- ❑ Using this formula, the algorithm is defined for the unbounded (0-1) knapsack problem with the unlimited number of items for each type.



5.1.3. (0-1) Knapsack problem

Step 3: Compute the value of an optimal solution in a *bottom-up* fashion

```
for i := 0 to M do cost[i] := 0;  
for j := 1 to N do /* each of item type */  
begin  
    for i := 1 to M do /* i means capacity */  
        if i - size[j] >= 0 then  
            if cost[i] < (cost[i - size[j]] + val[j]) then  
                begin  
                    cost[i] := cost[i - size[j]] + val[j];  
                    best[i] := j  
                end;  
        end;  
end;
```



5.1.3. (0-1) Knapsack problem

Step 4: Construct the *final optimal solution* from computed information

- The final optimal solution is stored in $cost[M]$ and $best[M]$ for the maximum total value of the knapsack and the last item added into the knapsack corresponding to the maximum value.
- Other items are then derived iteratively by subtracting *the total value* down to 0.

```
kv := cost[M]; s := M;  
while (kv > 0) begin  
    print best[s];  
    kv := kv - val[best[s]]; s := s - size[best[s];  
end;
```



- [illegible]

val[E] = 13



5.1.3. (0-1) Knapsack problem

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
<i>A</i>	<i>size</i>	=	3		<i>val</i>	=	4										
cost	0	0	4	4	4	8	8	8	12	12	12	16	16	16	20	20	20
best	-	-	A	A	A	A	A	A	A	A	A	A	A	A	A	A	A
<i>B</i>	<i>size</i>	=	4		<i>val</i>	=	5										
cost	0	0	4	5	5	8	9	10	12	13	14	16	17	18	20	21	22
best	-	-	A	B	B	A	B	B	A	B	B	A	B	B	A	B	B
<i>C</i>	<i>size</i>	=	7		<i>val</i>	=	10										
cost	0	0	4	5	5	8	10	10	12	14	15	16	18	20	20	22	24
best	-	-	A	B	B	A	C	B	A	C	C	A	C	C	A	C	C
<i>D</i>	<i>size</i>	=	8		<i>val</i>	=	11										
cost	0	0	4	5	5	8	10	11	12	14	15	16	18	20	21	22	24
best	-	-	A	B	B	A	C	D	A	C	C	A	C	C	D	C	C
<i>E</i>	<i>size</i>	=	9		<i>val</i>	=	13										
cost	0	0	4	5	5	8	10	11	13	14	15	17	18	20	21	23	24
best	-	-	A	B	B	A	C	D	E	C	C	E	C	C	D	E	C



5.1.3. (0-1) Knapsack problem

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
<i>A</i>	<i>size</i>	=	3		<i>val</i>	=	4										
cost	0	0	4	4	4	8	8	8	12	12	12	16	16	16	20	20	20
best	-	-	A	A	A	A	A	A	A	A	A	A	A	A	A	A	A
<i>B</i>	<i>size</i>	=	4		<i>val</i>	=	5										
cost	0	0	4	5	5	8	9	10	12	13	14	16	17	18	20	21	22
best	-	-	A	B	B	A	B	B	A	B	B	A	B	B	A	B	B
<i>C</i>	<i>size</i>	=	Before considering B: $\text{cost}[8] = 8, \text{best}[8] = A$ Consider B: $\text{cost}[8] = \text{cost}[8 - \text{size}[B]] + \text{val}[B]$ $\text{cost}[8] = \text{cost}[4] + 5 = 5 + 5 = 10$ After considering B: $\text{cost}[8] = 10, \text{best}[8] = B$														
cost	0	0												20	20	22	24
best	-	-												C	A	C	C
<i>D</i>	<i>size</i>	=															
cost	0	0												20	21	22	24
best	-	-												C	D	C	C
<i>E</i>	<i>size</i>	=															
cost	0	0												20	21	23	24
best	-	-												C	D	E	C



5.1.3. (0-1) Knapsack problem

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
<i>A</i>	<i>size</i>	=	3		<i>val</i>	=	4										
cost	0	0	4	4	4	8	8	8	12	12	12	16	16	16	20	20	20
best	-	-	A	A	A	A	A	A	A	A	A	A	A	A	A	A	A
<i>B</i>	<i>size</i>	=	4		<i>val</i>	=	5										
cost	0	0	4	5	5	8	9	10	12	13	14	16	17	18	20	21	22
best	-	-	A	B	B	A	B	B	A	B	B	A	B	B	A	B	B
<i>C</i>	<i>size</i>	=	Before considering B: $\text{cost}[9] = \mathbf{12}$, $\text{best}[9] = A$ Consider B: $\text{cost}[9] = \text{cost}[9 - \text{size}[B]] + \text{val}[B]$ $\text{cost}[9] = \text{cost}[5] + 5 = 5 + 5 = 10$ After considering B: $\text{cost}[9] = \mathbf{12}$, $\text{best}[9] = A$														
cost	0	0												20	20	22	24
best	-	-												C	A	C	C
<i>D</i>	<i>size</i>	=															
cost	0	0												20	21	22	24
best	-	-												C	D	C	C
<i>E</i>	<i>size</i>	=															
cost	0	0												20	21	23	24
best	-	-												C	D	E	C



5.1.3. (0-1) Knapsack problem

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
<i>A</i>	<i>size</i>	=	3		<i>val</i>	=	4										
cost	0	0	4	4	4	8	8	8	12	12	12	16	16	16	20	20	20
best	-	-	A	A	A	A	A	A	A	A	A	A	A	A	A	A	A
<i>B</i>	<i>size</i>	=	4		<i>val</i>	=	5										
cost	0	0	4	5	5	8	9	10	12	13	14	16	17	18	20	21	22
best	-	-	A	B	B	A	B	B	A	B	B	A	B	B	A	B	B
<i>C</i>	<i>size</i>	=															
cost	0	0													20	20	22
best	-	-													C	A	C
<i>D</i>	<i>size</i>	=															
cost	0	0													20	21	22
best	-	-													C	D	C
<i>E</i>	<i>size</i>	=															
cost	0	0													20	21	23
best	-	-													C	D	E

Before considering B:

$$\text{cost}[10] = 12, \text{best}[10] = A$$

Consider B:

$$\text{cost}[10] = \text{cost}[10 - \text{size}[B]] + \text{val}[B]$$

$$\text{cost}[10] = \text{cost}[6] + 5 = 8 + 5 = \mathbf{13}$$

After considering B:

$$\text{cost}[10] = \mathbf{13}, \text{best}[10] = B$$



5.1.3. (0-1) Knapsack problem

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
<i>A</i>	<i>size</i>	=	3		<i>val</i>	=	4										
cost	0	0	4	4	4	8	8	8	12	12	12	16	16	16	20	20	20
best	-	-	A	A	A	A	A	A	A	A	A	A	A	A	A	A	A
<i>B</i>	<i>size</i>	=	4		<i>val</i>	=	5										
cost	0	0	4	5	5	8	9	10	12	13	14	16	17	18	20	21	22
best	-	-	A	B	B	A	B	B	A	B	B	A	B	B	A	B	B
<i>C</i>	<i>size</i>	=	7		<i>val</i>	=	10										
cost	0	0	4	5	5	8	10	10	12	14	15	16	18	20	20	22	24
best	-	-	A	B	B	A	C	B	A	C	C	A	C	C	A	C	C
<i>D</i>	<i>size</i>	=	8		<i>val</i>	=	11										
cost	0	0	4	5	5	8	10	11	12	14	15	16	18	20	21	22	24
best	-	-	A	B	B	A	C	D	A	C	C	A	C	C	D	C	C
<i>E</i>	<i>size</i>	=	9		<i>val</i>	=	13										
cost	0	0	4	5	5	8	10	11	13	14	15	17	18	20	21	23	24
best	-	-	A	B	B	A	C	D	E	C	C	E	C	C	D	E	C



5.1.3. (0-1) Knapsack problem

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
<i>A</i>	<i>size</i>	=	3		<i>val</i>	=	4										
cost	0	0	4	4	4	8	8	8	12	12	12	16	16	16	20	20	20
best	-	-	A	A	A	A	A	A	A	A	A	A	A	A	A	A	A
<i>B</i>	<i>size</i>	=	4		<i>val</i>	=	5										
cost	0	0	4	5	5	8	9	10	12	13	14	16	17	18	20	21	22
best	-	-	A	B	B	A	B	B	A	B	B	A	B	B	A	B	B
<i>C</i>	<i>size</i>	=	7		<i>val</i>	=	10										
cost	0	0	4	5	5	8	10	10	12	14	15	16	18	20	20	22	24
best	-	-	A	B	B	A	C	B	A	C	C	A	C	C	A	C	C
<i>D</i>	<i>size</i>	=	8		<i>val</i>	=	11										
cost	0	0	4	5	5	8	10	11	12	14	15	16	18	20	21	22	24
best	-	-	A	B	B	A	C	D	A	C	C	A	C	C	D	C	C
<i>E</i>	<i>size</i>	=	9		<i>val</i>	=	13										
cost	0	0	4	5	5	8	10	11	13	14	15	17	18	20	21	23	24
best	-	-	A	B	B	A	C	D	E	C	C	E	C	C	D	E	C



5.1.3. (0-1) Knapsack problem

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
<i>A</i>	<i>size</i>	=	3		<i>val</i>	=	4										
cost	0	0	4	4	4	8	8	8	12	12	12	16	16	16	20	20	20
best	-	-	A	A	A	A	A	A	A	A	A	A	A	A	A	A	A
<i>B</i>	<i>size</i>	=	4		<i>val</i>	=	5										
cost	0	0	4	5	5	8	9	10	12	13	14	16	17	18	20	21	22
best	-	-	A	B	B	A	B	B	A	B	B	A	B	B	A	B	B
<i>C</i>	<i>size</i>	=	7		<i>val</i>	=	10										
cost	0	0	4	5	5	8	10	10	12	14	15	16	18	20	20	22	24
best	-	-	A	B	B	A	C	B	A	C	C	A	C	C	A	C	C
<i>D</i>	<i>size</i>	=	8		<i>val</i>	=	11										
cost	0	0	4	5	5	8	10	11	12	14	15	16	18	20	21	22	24
best	-	-	A	B	B	A	C	D	A	C	C	A	C	C	D	C	C
<i>E</i>	<i>size</i>	=	9		<i>val</i>	=	13										
cost	0	0	4	5	5	8	10	11	13	14	15	17	18	20	21	23	24
best	-	-	A	B	B	A	C	D	E	C	C	E	C	C	D	E	C



- [illegible]

val[E] = 13

Combination of items = {**C**, **C**, **A**}



5.1.3. (0-1) Knapsack problem

- Time complexity of finding the combination of items of N types that maximizes the total value of the knapsack M
 - Abstract operation: *assignment*
 - Worst case:

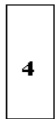
$$C_{\text{knapsack}} = 2MN + M = O(MN)$$

- Note: the algorithm works with positive *integers* for M and each size. It is (unbounded) 0-1 knapsack problem.



5.1.3. (0-1) Knapsack problem

- “A thief robbing a store finds it filled with N types of items of varying size and value, but has only a small knapsack of capacity M to use to carry the goods. *The knapsack problem* is to find the combination of items which the thief should choose for his knapsack in order to maximize the total value of all the items he takes.”

					
name	A	B	C	D	E
size	3	4	7	8	9
value	4	5	10	11	13

Knapsack capacity: $M = 17$

Assume that each type has *only one* item.

What is the *combination of items* that *maximizes* the total value of the knapsack?



5.1.3. (0-1) Knapsack problem

- A dynamic programming algorithm
 - Extended from the previous one with tables $cost[N+1, M+1]$ and $best[N+1, M+1]$ instead of arrays $cost[M+1]$ and $best[M+1]$

	0		$j-size[i]$		j		M
0	0	...	0	...	0	...	0
...	...						
$i-1$	0	...	$cost[i-1, j-size[i]]$	...	$cost[i-1, j]$	...	...
i	0				$cost[i, j]$		
...	...						...
N	0					...	<i>goal</i>

$$cost[i, j] = \begin{cases} \max\{cost[i-1, j], cost[i-1, j-size[i]] + value[i]\} & \text{if } j-size[i] \geq 0 \\ cost[i-1, j] & \text{if } j-size[i] < 0 \end{cases}$$



5.1.3. (0-1) Knapsack problem

- A dynamic programming algorithm for the bounded (0-1) Knapsack problem with *only one item* for each type

Time complexity:

$O(MN)$

```
for j := 0 to M do cost[0, j] := 0;
for i := 0 to N do cost[i, 0] := 0;
for i := 1 to N do /* each of item type */
begin
  for j := 1 to M do /* j means capacity */
    cost[i, j] := cost[i-1, j];
    best[i, j] := best[i-1, j];
    if j - size[i] >= 0 then
      if cost[i-1, j] < (cost[i-1, j-size[i]] + val[i]) then
        begin
          cost[i, j] := cost[i-1, j-size[i]] + val[i];
          best[i, j] := i;
        end;
    end;
  end;
end;
```



- $N = 5$, $M = 17$, **only one item** for each type

```
val[A] = 4
val[B] = 5
val[C] = 10
val[D] = 11
val[E] = 13
```



5.1.3. (0-1) Knapsack problem

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
<i>A</i>	<i>size</i>	=	3		<i>val</i>	=	4										
cost	0	0	4	4	4	4	4	4	4	4	4	4	4	4	4	4	4
best	-	-	A	A	A	A	A	A	A	A	A	A	A	A	A	A	A
<i>B</i>	<i>size</i>	=	4		<i>val</i>	=	5										
cost	0	0	4	5	5	5	9	9	9	9	9	9	9	9	9	9	9
best	-	-	A	B	B	B	B	B	B	B	B	B	B	B	B	B	B
<i>C</i>	<i>size</i>	=	7		<i>val</i>	=	10										
cost	0	0	4	5	5	5	10	10	10	14	15	15	15	19	19	19	19
best	-	-	A	B	B	B	C	C	C	C	C	C	C	C	C	C	C
<i>D</i>	<i>size</i>	=	8		<i>val</i>	=	11										
cost	0	0	4	5	5	5	10	11	11	14	15	16	16	19	21	21	21
best	-	-	A	B	B	B	C	D	D	C	C	D	D	C	D	D	D
<i>E</i>	<i>size</i>	=	9		<i>val</i>	=	13										
cost	0	0	4	5	5	5	10	11	13	14	15	17	18	19	21	23	24
best	-	-	A	B	B	B	C	D	E	C	C	E	E	C	D	E	E



5.1.3. (0-1) Knapsack problem

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
<i>A</i>	<i>size</i>	=	3		<i>val</i>	=	4										
cost	0	0	4	4	4	4	4	4	4	4	4	4	4	4	4	4	4
best	-	-	A	A	A	A	A	A	A	A	A	A	A	A	A	A	A
<i>B</i>	<i>size</i>	=	4		<i>val</i>	=	5										
cost	0	0	4	5	5	5	9	9	9	9	9	9	9	9	9	9	9
best	-	-	A	B	B	B	B	B	B	B	B	B	B	B	B	B	B
<i>C</i>	<i>size</i>	=															
cost	0	0													19	19	19
best	-	-													C	C	C
<i>D</i>	<i>size</i>	=															
cost	0	0													21	21	21
best	-	-													D	D	D
<i>E</i>	<i>size</i>	=															
cost	0	0													21	23	24
best	-	-													D	E	E

Before considering B:

$$\text{cost}[1, 6] = 4, \text{best}[1, 6] = A$$

Consider B:

$$\text{cost}[2, 6] = \text{cost}[1, 6 - \text{size}[B]] + \text{val}[B]$$

$$\text{cost}[2, 6] = \text{cost}[1, 2] + 5 = 0 + 5 = \mathbf{5}$$

After considering B:

$$\text{cost}[2, 6] = \mathbf{5}, \text{best}[2, 6] = B$$



	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
A	size	=	3		val	=	4										
cost	0	0	4	4	4	4	4	4	4	4	4	4	4	4	4	4	4
best	-	-	A	A	A	A	A	A	A	A	A	A	A	A	A	A	A
B	size	=	4		val	=	5										
cost	0	0	4	5	5	5	9	9	9	9	9	9	9	9	9	9	9
best	-	-	A	B	B	B	B	B	B	B	B	B	B	B	B	B	B
C	size	=	Before considering B: cost[1, 7] = 4, best[1, 7] = A														
cost	0	0													19	19	19
best	-	-													C	C	C
D	size	=	Consider B: cost[2, 7] = cost[1, 7-size[B]] + val[B] cost[2, 7] = cost[1, 3] + 5 = 4 + 5 = 9														
cost	0	0													21	21	21
best	-	-													D	D	D
E	size	=	After considering B: cost[2, 7] = 9, best[2, 7] = B														
cost	0	0													21	23	24
best	-	-													D	E	E



5.1.3. (0-1) Knapsack problem

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
<i>A</i>	<i>size</i>	=	3		<i>val</i>	=	4										
cost	0	0	4	4	4	4	4	4	4	4	4	4	4	4	4	4	4
best	-	-	A	A	A	A	A	A	A	A	A	A	A	A	A	A	A
<i>B</i>	<i>size</i>	=	4		<i>val</i>	=	5										
cost	0	0	4	5	5	5	9	9	9	9	9	9	9	9	9	9	9
best	-	-	A	B	B	B	B	B	B	B	B	B	B	B	B	B	B
<i>C</i>	<i>size</i>	=															
cost	0	0													19	19	19
best	-	-													C	C	C
<i>D</i>	<i>size</i>	=															
cost	0	0													21	21	21
best	-	-													D	D	D
<i>E</i>	<i>size</i>	=															
cost	0	0													21	23	24
best	-	-													D	E	E

Before considering B:

$$\text{cost}[1, 8] = 4, \text{best}[1, 8] = A$$

Consider B:

$$\text{cost}[2, 8] = \text{cost}[1, 8 - \text{size}[B]] + \text{val}[B]$$

$$\text{cost}[2, 8] = \text{cost}[1, 4] + 5 = 4 + 5 = 9$$

After considering B:

$$\text{cost}[2, 8] = 9, \text{best}[2, 8] = B$$



5.1.3. (0-1) Knapsack problem

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
<i>A</i>	<i>size</i>	=	3		<i>val</i>	=	4										
cost	0	0	4	4	4	4	4	4	4	4	4	4	4	4	4	4	4
best	-	-	A	A	A	A	A	A	A	A	A	A	A	A	A	A	A
<i>B</i>	<i>size</i>	=	4		<i>val</i>	=	5										
cost	0	0	4	5	5	5	9	9	9	9	9	9	9	9	9	9	9
best	-	-	A	B	B	B	B	B	B	B	B	B	B	B	B	B	B
<i>C</i>	<i>size</i>	=	7		<i>val</i>	=	10										
cost	0	0	4	5	5	5	10	10	10	14	15	15	15	19	19	19	19
best	-	-	A	B	B	B	C	C	C	C	C	C	C	C	C	C	C
<i>D</i>	<i>size</i>	=	8		<i>val</i>	=	11										
cost	0	0	4	5	5	5	10	11	11	14	15	16	16	19	21	21	21
best	-	-	A	B	B	B	C	D	D	C	C	D	D	C	D	D	D
<i>E</i>	<i>size</i>	=	9		<i>val</i>	=	13										
cost	0	0	4	5	5	5	10	11	13	14	15	17	18	19	21	23	24
best	-	-	A	B	B	B	C	D	E	C	C	E	E	C	D	E	E



5.1.3. (0-1) Knapsack problem

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
<i>A</i>	<i>size</i>	=	3		<i>val</i>	=	4										
cost	0	0	4	4	4	4	4	4	4	4	4	4	4	4	4	4	4
best	-	-	A	A	A	A	A	A	A	A	A	A	A	A	A	A	A
<i>B</i>	<i>size</i>	=	4		<i>val</i>	=	5										
cost	0	0	4	5	5	5	9	9	9	9	9	9	9	9	9	9	9
best	-	-	A	B	B	B	B	B	B	B	B	B	B	B	B	B	B
<i>C</i>	<i>size</i>	=	7		<i>val</i>	=	10										
cost	0	0	4	5	5	5	10	10	10	14	15	15	15	19	19	19	19
best	-	-	A	B	B	B	C	C	C	C	C	C	C	C	C	C	C
<i>D</i>	<i>size</i>	=	8		<i>val</i>	=	11										
cost	0	0	4	5	5	5	10	11	11	14	15	16	16	19	21	21	21
best	-	-	A	B	B	B	C	D	D	C	C	D	D	C	D	D	D
<i>E</i>	<i>size</i>	=	9		<i>val</i>	=	13										
cost	0	0	4	5	5	5	10	11	13	14	15	17	18	19	21	23	24
best	-	-	A	B	B	B	C	D	E	C	C	E	E	C	D	E	E



- [illegible]

val[E] = 13

Combination of items = {**E**, **D**}



5.1.3. (0-1) Knapsack problem

Your turn:

- Trace-by-hand both algorithms for the unbounded and bounded (0-1) knapsack problems as follows.

- 5.1.3.1. $N = 4, M = 10$

size[A] = 5, val[A] = 9, size[B] = 4, val[B] = 8, size[C] = 2, val[C] = 3, size[D] = 3, val[D] = 5

- 5.1.3.2. $N = 5, M = 6$

size[A] = 3, val[A] = 25, size[B] = 2, val[B] = 20, size[C] = 1, val[C] = 15, size[D] = 4, val[D] = 40, size[E] = 5, val[E] = 50

- 5.1.3.3. $N = 4, M = 8$

size[A] = 2, val[A] = 4, size[B] = 3, val[B] = 7, size[C] = 5, val[C] = 9, size[D] = 6, val[D] = 11

- 5.1.3.4. $N = 4, M = 9$

size[A] = 2, val[A] = 3, size[B] = 3, val[B] = 4, size[C] = 4, val[C] = 5, size[D] = 5, val[D] = 7



5.1.3. (0-1) Knapsack problem

Your turn:

- ❑ 5.1.3.5. Design and analyze the algorithm that outputs the items packed in the knapsack after *cost* and *best* are obtained from the dynamic programming algorithm when there is ***only one item*** for each type.
- ❑ 5.1.3.6. Design and analyze the algorithm that handles the (0-1) knapsack problem with a limited number of items for each type.

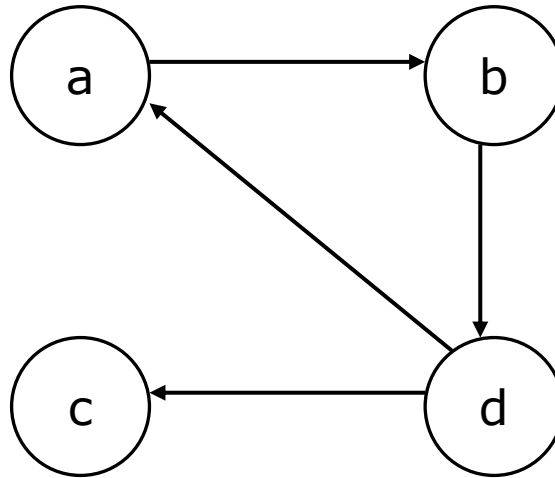


5.1.4. Transitive closure with Warshall's algorithm

- ❑ For directed graphs, we're often interested in the set of vertices that can be reached from a given vertex by traversing edges from the graph in the indicated direction.
- ❑ One operation we might want to perform is “to add an edge directly from vertex x to vertex y if there is some way to get from x to y ”.
- ❑ The graph that is obtained by adding all edges of this nature to a directed graph is called the *transitive closure* of the graph.
- ❑ Since the transitive closure is likely to be *dense*, so an *adjacency matrix representation* is called for.

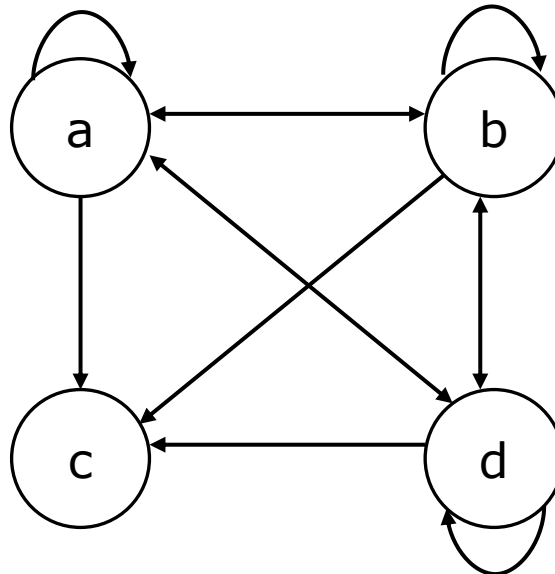
5.1.4. Transitive closure with Warshall's algorithm

A directed graph
and
its adjacency matrix



	a	b	c	d
a	0	1	0	0
b	0	0	0	1
c	0	0	0	0
d	1	0	1	0

and
its transitive closure



	a	b	c	d
a	1	1	1	1
b	1	1	1	1
c	0	0	0	0
d	1	1	1	1



5.1.4. Transitive closure with Warshall's algorithm

□ Warshall's algorithm

There is a simple algorithm for computing the transitive closure of a graph represented by an adjacency matrix.

```
for  $y := 1$  to  $V$  do  
  for  $x := 1$  to  $V$  do  
    if  $a[x, y]$  then  
      for  $j := 1$  to  $V$  do  
        if  $a[y, j]$  then  $a[x, j] := \text{true}; // \text{true} = 1$ 
```

S. Warshall invented this method in 1962, using the simple observation: *“If there is a way to get from node x to node y and a way to get from node y to node j , then there is a way to get from node x to node j .”*



5.1.4. Transitive closure with Warshall's algorithm

- Warshall's algorithm repeats V iterations on the adjacency matrix a , constructing a series of V boolean matrices:

$$a^{(0)}, \dots, a^{(y-1)}, a^{(y)}, \dots, a^{(V)}$$

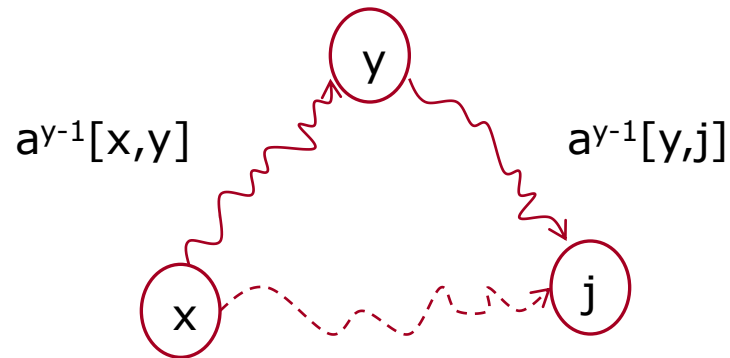
- The central point of the algorithm is that we can compute all elements of each matrix $a^{(y)}$ from its immediate predecessor $a^{(y-1)}$ in series.
- After the y -th iteration, $a[x, j]$ is equal to 1 if and only if there exists a directed path of a positive length from vertex x to vertex j with each *intermediate vertex*, if any, numbered not higher than y .
- After the y -th iteration, we compute the elements of matrix a by the following formula:

$$a^y[x, j] = a^{y-1}[x, j] \text{ or } (a^{y-1}[x, y] \text{ and } a^{y-1}[y, j])$$

- Warshall's algorithm applies dynamic programming paradigm since it uses the recurrence formula but it does not bring out a recursive algorithm. Instead, it brings out an iterative algorithm with the support of a matrix for storing intermediate results.

5.1.4. Transitive closure with Warshall's algorithm

- Rule for changing zeros in Warshall's algorithm

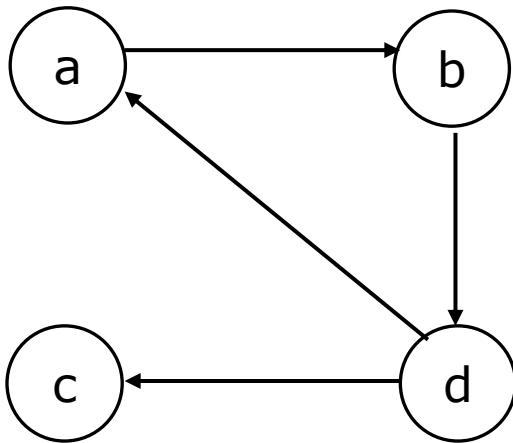


$$a^y[x,j] = a^{y-1}[x,j] \text{ or } (a^{y-1}[x,y] \text{ and } a^{y-1}[y,j])$$

$$a^{y-1} = \begin{matrix} & j & y \\ y & \begin{bmatrix} & \\ & \end{bmatrix} \\ x & \begin{bmatrix} & \end{bmatrix} \end{matrix} \begin{matrix} \uparrow 0 \rightarrow 1 \end{matrix} \rightarrow a^y = \begin{matrix} & j & y \\ y & \begin{bmatrix} 1 & \\ & \end{bmatrix} \\ x & \begin{bmatrix} \mathbf{1} & 1 \end{bmatrix} \end{matrix}$$

5.1.4. Transitive closure with Warshall's algorithm

Given a directed graph, what is its transitive closure?

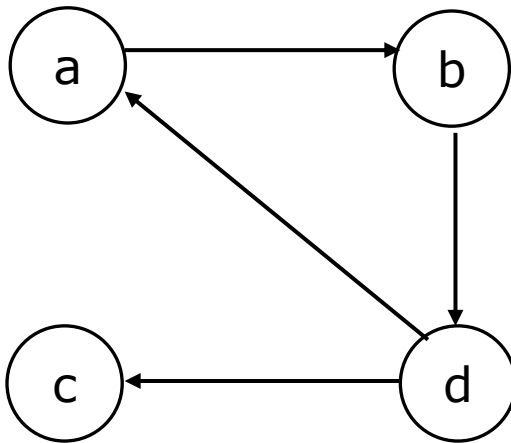


$A^0 =$

	a	b	c	d
a	0	1	0	0
b	0	0	0	1
c	0	0	0	0
d	1	0	1	0

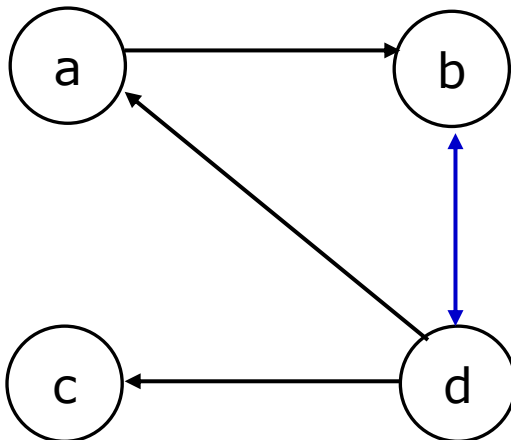
5.1.4. Transitive closure with Warshall's algorithm

Given a directed graph, what is its transitive closure?



$A^0 =$

	a	b	c	d
a	0	1	0	0
b	0	0	0	1
c	0	0	0	0
d	1	0	1	0

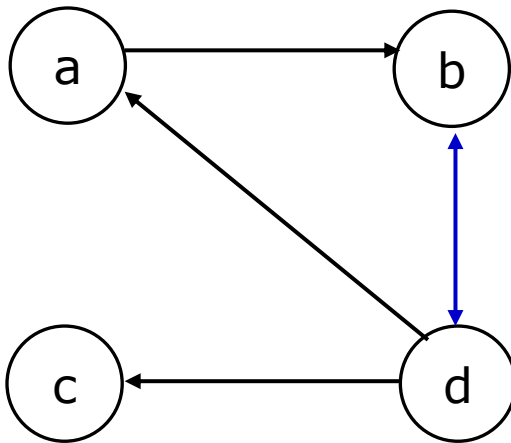


$A^1 =$

	a	b	c	d
a	0	1	0	0
b	0	0	0	1
c	0	0	0	0
d	1	1	1	0

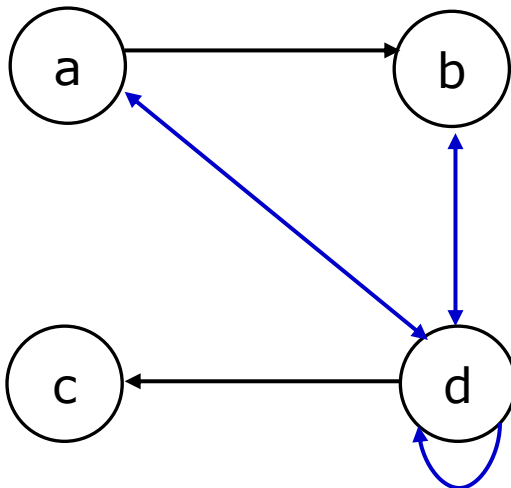
5.1.4. Transitive closure with Warshall's algorithm

Given a directed graph, what is its transitive closure?



$A^1 =$

	a	b	c	d
a	0	1	0	0
b	0	0	0	1
c	0	0	0	0
d	1	1	1	0

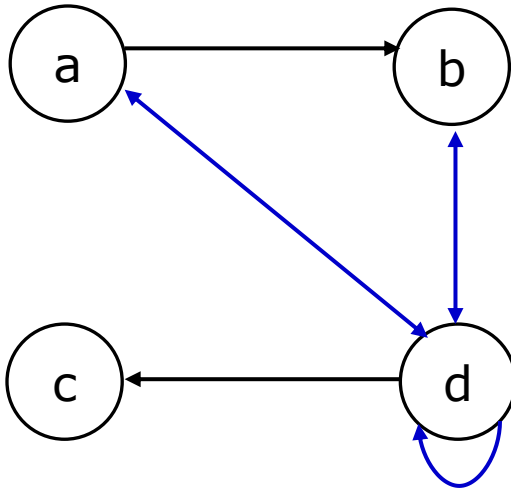


$A^2 =$

	a	b	c	d
a	0	1	0	1
b	0	0	0	1
c	0	0	0	0
d	1	1	1	1

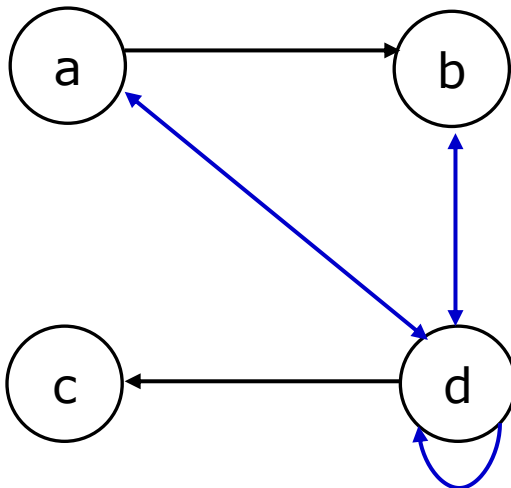
5.1.4. Transitive closure with Warshall's algorithm

Given a directed graph, what is its transitive closure?



$A^2 =$

	a	b	c	d
a	0	1	0	1
b	0	0	0	1
c	0	0	0	0
d	1	1	1	1

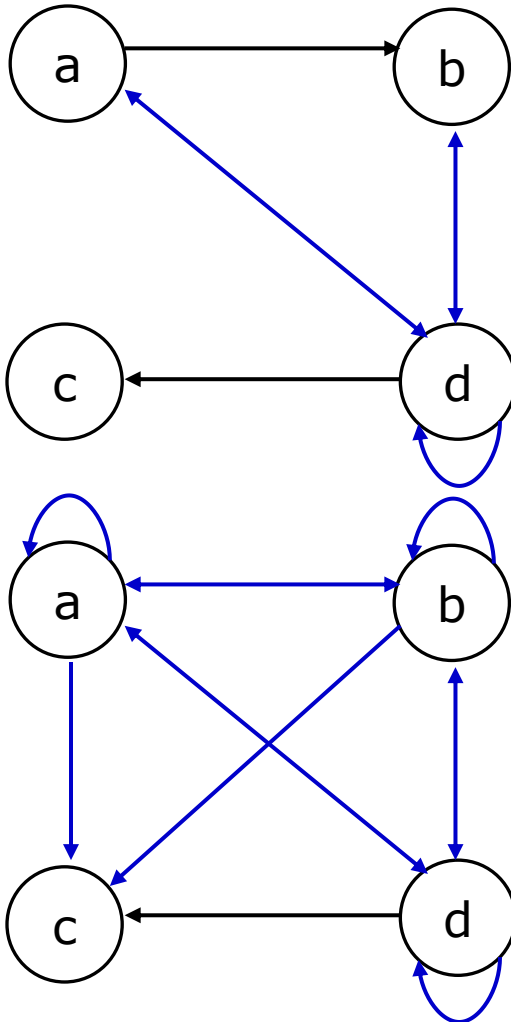


$A^3 =$

	a	b	c	d
a	0	1	0	1
b	0	0	0	1
c	0	0	0	0
d	1	1	1	1

5.1.4. Transitive closure with Warshall's algorithm

Given a directed graph, what is its transitive closure?



$A^3 =$

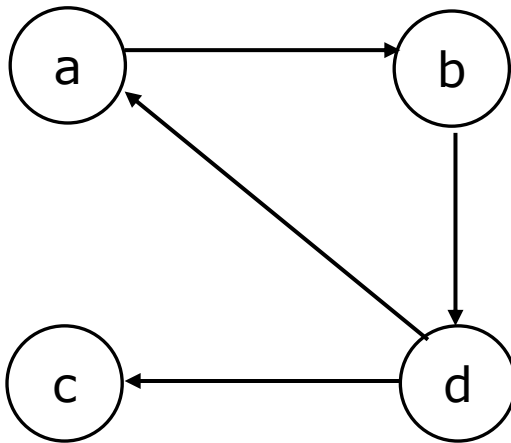
	a	b	c	d
a	0	1	0	1
b	0	0	0	1
c	0	0	0	0
d	1	1	1	1

$A^4 =$

	a	b	c	d
a	1	1	1	1
b	1	1	1	1
c	0	0	0	0
d	1	1	1	1

5.1.4. Transitive closure with Warshall's algorithm

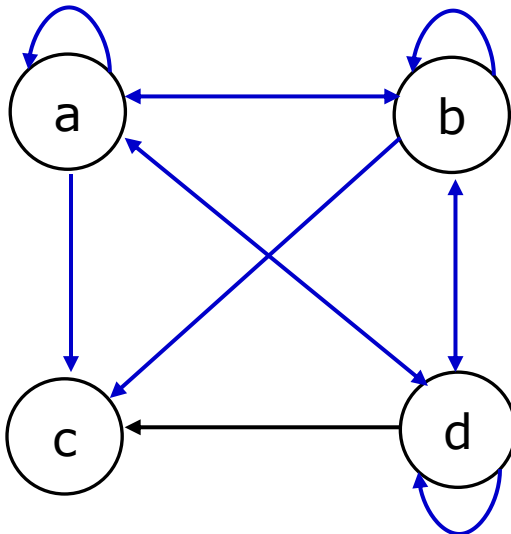
Given a directed graph, what is its transitive closure?



$A^0 =$

	a	b	c	d
a	0	1	0	0
b	0	0	0	1
c	0	0	0	0
d	1	0	1	0

Transitive closure



$A^4 =$

	a	b	c	d
a	1	1	1	1
b	1	1	1	1
c	0	0	0	0
d	1	1	1	1



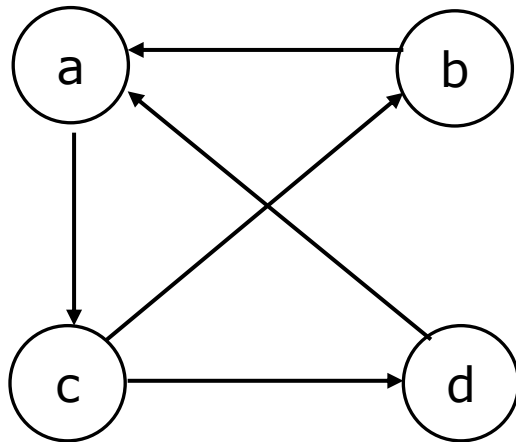
5.1.4. Transitive closure with Warshall's algorithm

- Time complexity of Warshall's algorithm to find transitive closure of a directed graph
 - Abstract operation: *assignment*
 - Worst case: $O(|V|^3)$
- Note:
 - The algorithm runs faster by treating matrix rows as bit strings and employing bitwise *or* operation.
 - No need of separate matrices for recording intermediate results.
 - The underlying idea can be applied to the more general problem of finding lengths of the shortest paths in weighted graphs.

5.1.4. Transitive closure with Warshall's algorithm

Your turn:

- For each directed graph, present both *graph* and *adjacency matrix*. Then, *trace-by-hand* Warshall's algorithm to find its transitive closure.



5.1.4.1.

	a	b	c	d
a	0	1	0	0
b	0	0	1	0
c	0	0	0	1
d	0	0	0	0

5.1.4.2.

	a	b	c	d
a	0	0	0	1
b	1	0	1	1
c	1	0	0	1
d	0	0	1	0

5.1.4.3.



5.1.4. Transitive closure with Warshall's algorithm

Your turn:

- ❑ 5.1.4.4. Explain how Warshall's algorithm can be used to determine whether a given digraph is a dag (directed acyclic graph). Is it a good algorithm for this problem?
- ❑ 5.1.4.5. Is it a good idea to apply Warshall's algorithm to find the transitive closure of an undirected graph? If yes, give an example to demonstrate it.

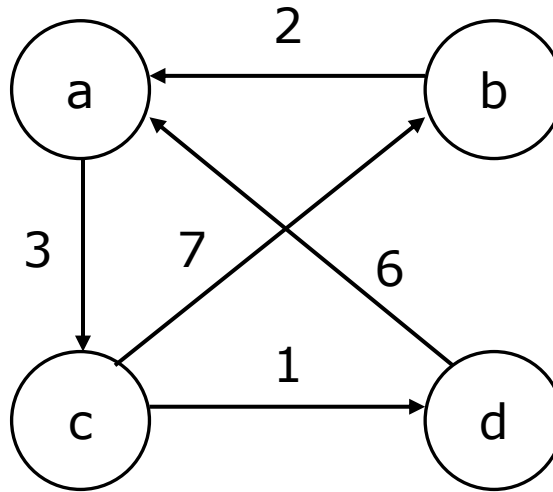


5.1.5. The all-pairs shortest path problem with Floyd's algorithm

- ❑ For weighted graphs (*directed* or not) one might want to build a matrix allowing one to find the shortest path from x to y for all pairs of vertices. This is the *all-pairs shortest path problem*.
- ❑ It is also possible to use the idea just like Warshall's algorithm, which is attributed to R. W. Floyd.
- ❑ Floyd's algorithm
 - A weighted graph is represented by an adjacency matrix.
 - Dynamic programming is applied for an iterative algorithm.
 - It is applicable to both undirected and directed weighted graphs provided that they do not contain a cycle of a negative length.

5.1.5. The all-pairs shortest path problem with Floyd's algorithm

A weighted graph
and
its adjacency matrix



	a	b	c	d
a	0	∞	3	∞
b	2	0	∞	∞
c	∞	7	0	1
d	6	∞	∞	0

and
its distance matrix
for the weights of
the shortest paths
and the matrix for
intermediate vertices

	a	b	c	d
a	0	10	3	4
b	2	0	5	6
c	7	7	0	1
d	6	16	9	0

	a	b	c	d
a	0	3	0	3
b	0	0	1	3
c	4	0	0	0
d	0	3	1	0



5.1.5. The all-pairs shortest path problem with Floyd's algorithm

□ Floyd's algorithm

```
for y := 1 to V do  
  for x := 1 to V do  
    if a [x, y] > 0 then  
      for j := 1 to V do  
        if a [y, j] > 0 then  
          if (a[x, y] + a[y, j] < a [x, j]) then  
            begin  
              a[x, j] := a[x, y] + a[y, j];  
            end;
```



5.1.5. The all-pairs shortest path problem with Floyd's algorithm

- Floyd's algorithm repeats V iterations on the adjacency matrix a , constructing a series of V matrices:

$$a^{(0)}, \dots, a^{(y-1)}, a^{(y)}, \dots, a^{(V)}$$

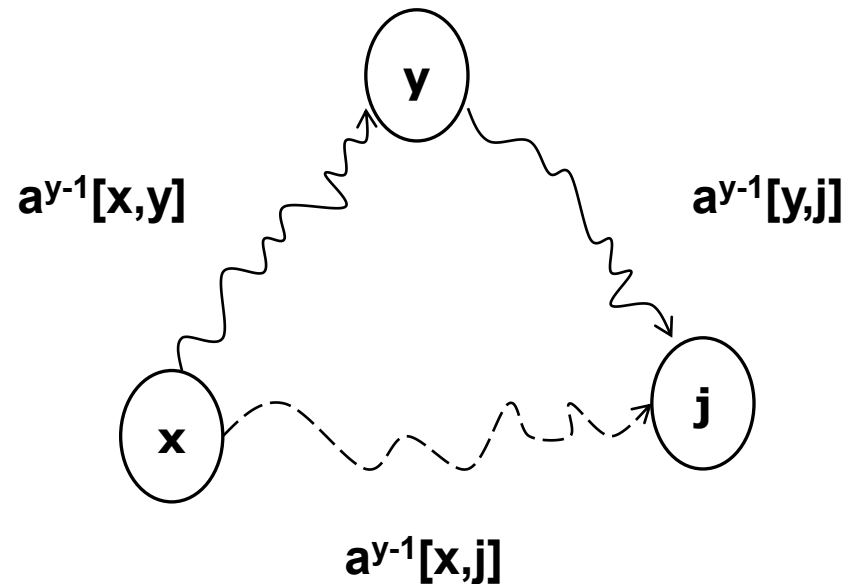
- The central point of the algorithm is that we can compute all elements of each matrix $a^{(y)}$ from its immediate predecessor $a^{(y-1)}$.
- After the y -th iteration, $a[x, j]$ stores the shortest length of the directed path from the vertex x to vertex j with each *intermediate vertex*, if any, numbered not higher than y .
- After the y -th iteration, we compute the elements of matrix a by the following formula:

$$a^y[x, j] = \min (a^{y-1}[x, j], a^{y-1}[x, y] + a^{y-1}[y, j])$$

- The superscript y indicates the value of an element in matrix a after the y -th iteration.

5.1.5. The all-pairs shortest path problem with Floyd's algorithm

□ Floyd's algorithm



$$a^y[x,j] = \min (a^{y-1}[x,j], a^{y-1}[x, y] + a^{y-1}[y, j])$$



5.1.5. The all-pairs shortest path problem with Floyd's algorithm

□ Floyd's algorithm

Extended with matrix P , where $P[x, j]$ holds the vertex y that led Floyd's algorithm to find the smallest value of $a[x, j]$

```
for  $i := 1$  to  $V$  do
  for  $j := 1$  to  $V$  do
     $P[i, j] := 0$ ;
  for  $y := 1$  to  $V$  do
    for  $x := 1$  to  $V$  do
      if  $a[x, y] > 0$  then
        for  $j := 1$  to  $V$  do
          if  $a[y, j] > 0$  then
            if  $(a[x, y] + a[y, j] < a[x, j])$  then
              begin
                 $a[x, j] := a[x, y] + a[y, j]$ ;
                 $P[x, j] := y$ ;
              end;
```



5.1.5. The all-pairs shortest path problem with Floyd's algorithm

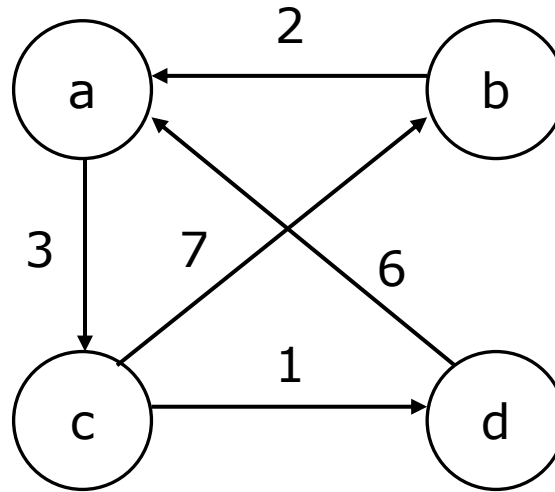
□ Floyd's algorithm

- Using matrix P and procedure $path()$, we can print the intermediate vertices y on the shortest path from vertex x to vertex j to *form the shortest path from x to j .*

```
procedure path(x, j: int)
var y : int;
begin
    y := P[x,j];
    if y = 0 then return;
    path(x,y); writeln(y); path(y,j);
end
```

5.1.5. The all-pairs shortest path problem with Floyd's algorithm

A weighted graph
and
its adjacency matrix



A^0

	a	b	c	d
a	0	∞	3	∞
b	2	0	∞	∞
c	∞	7	0	1
d	6	∞	∞	0

A^4

	a	b	c	d
a	0	10	3	4
b	2	0	5	6
c	7	7	0	1
d	6	16	9	0

p^4

	a	b	c	d
a	0	3	0	3
b	0	0	1	3
c	4	0	0	0
d	0	3	1	0

How to find
its distance matrix
for the weights of
the shortest paths
and the matrix for
intermediate vertices?



5.1.5. The all-pairs shortest path problem with Floyd's algorithm

A^0

	a	b	c	d
a	0	∞	3	∞
b	2	0	∞	∞
c	∞	7	0	1
d	6	∞	∞	0

P^0

	a	b	c	d
a	0	0	0	0
b	0	0	0	0
c	0	0	0	0
d	0	0	0	0

A^1

	a	b	c	d
a	0	∞	3	∞
b	2	0	5	∞
c	∞	7	0	1
d	6	∞	9	0

P^1

	a	b	c	d
a	0	0	0	0
b	0	0	1	0
c	0	0	0	0
d	0	0	1	0

From A^0 to A^1 :

From P^0 to P^1 :

5.1.5. The all-pairs shortest path problem with Floyd's algorithm

A^1

	a	b	c	d
a	0	∞	3	∞
b	2	0	5	∞
c	∞	7	0	1
d	6	∞	9	0

P^1

	a	b	c	d
a	0	0	0	0
b	0	0	1	0
c	0	0	0	0
d	0	0	1	0

A^2

	a	b	c	d
a	0	∞	3	∞
b	2	0	5	∞
c	9	7	0	1
d	6	∞	9	0

P^2

	a	b	c	d
a	0	0	0	0
b	0	0	1	0
c	2	0	0	0
d	0	0	1	0

From A^1 to A^2 :

From P^1 to P^2 :

5.1.5. The all-pairs shortest path problem with Floyd's algorithm

A^2

	a	b	c	d
a	0	∞	3	∞
b	2	0	5	∞
c	9	7	0	1
d	6	∞	9	0

P^2

	a	b	c	d
a	0	0	0	0
b	0	0	1	0
c	2	0	0	0
d	0	0	1	0

A^3

	a	b	c	d
a	0	10	3	4
b	2	0	5	6
c	9	7	0	1
d	6	16	9	0

P^3

	a	b	c	d
a	0	3	0	3
b	0	0	1	3
c	2	0	0	0
d	0	3	1	0

From A^2 to A^3 :

From P^2 to P^3 :

5.1.5. The all-pairs shortest path problem with Floyd's algorithm

A^3

	a	b	c	d
a	0	10	3	4
b	2	0	5	6
c	9	7	0	1
d	6	16	9	0

P^3

	a	b	c	d
a	0	3	0	3
b	0	0	1	3
c	2	0	0	0
d	0	3	1	0

A^4

	a	b	c	d
a	0	10	3	4
b	2	0	5	6
c	7	7	0	1
d	6	16	9	0

P^4

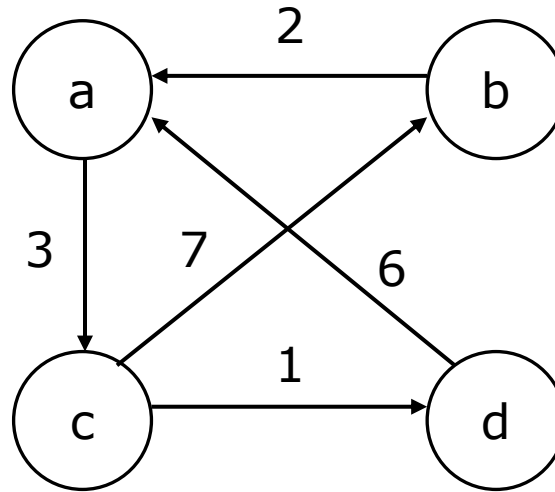
	a	b	c	d
a	0	3	0	3
b	0	0	1	3
c	4	0	0	0
d	0	3	1	0

From A^3 to A^4 :

From P^3 to P^4 :

5.1.5. The all-pairs shortest path problem with Floyd's algorithm

A weighted graph
and
its adjacency matrix



A^0

	a	b	c	d
a	0	∞	3	∞
b	2	0	∞	∞
c	∞	7	0	1
d	6	∞	∞	0

and
its distance matrix
for the weights of
the shortest paths
and the matrix for
intermediate vertices

A^4

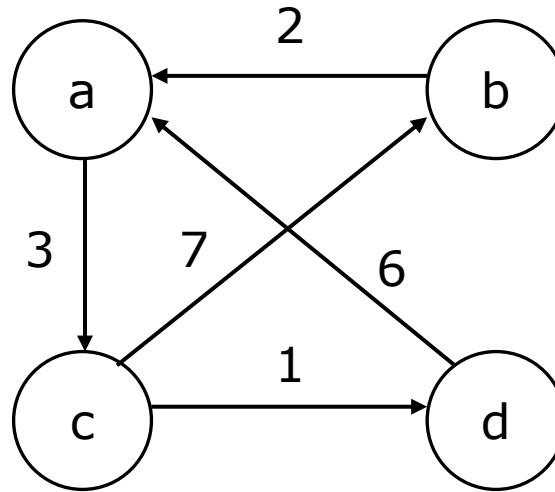
	a	b	c	d
a	0	10	3	4
b	2	0	5	6
c	7	7	0	1
d	6	16	9	0

P^4

	a	b	c	d
a	0	3	0	3
b	0	0	1	3
c	4	0	0	0
d	0	3	1	0

5.1.5. The all-pairs shortest path problem with Floyd's algorithm

A weighted graph
and
its adjacency matrix



A^0

	a	b	c	d
a	0	∞	3	∞
b	2	0	∞	∞
c	∞	7	0	1
d	6	∞	∞	0

A^4

	a	b	c	d
a	0	10	3	4
b	2	0	5	6
c	7	7	0	1
d	6	16	9	0

p^4

	a	b	c	d
a	0	3	0	3
b	0	0	1	3
c	4	0	0	0
d	0	3	1	0

What is the shortest
path from c to a?

→ path(c,a)

→ path(c,d), d,

path(d,a)

→ c, d, a



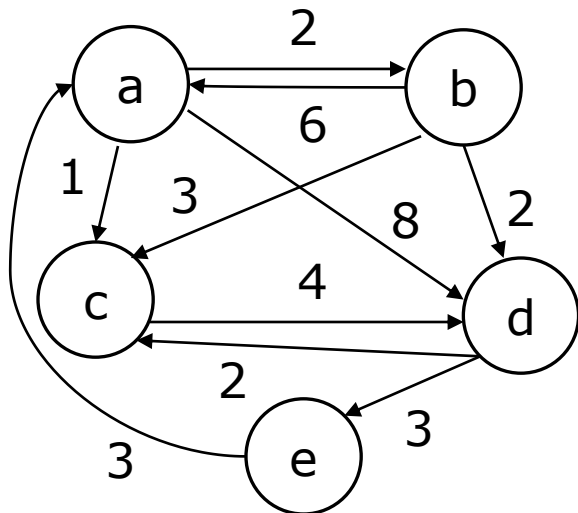
5.1.5. The all-pairs shortest path problem with Floyd's algorithm

- Time complexity of Floyd's algorithm for the all-pairs shortest path problem on a weighted graph of V vertices and E edges
 - Abstract operation: *assignment*
 - Worst case: $O(|V|^3)$

5.1.5. The all-pairs shortest path problem with Floyd's algorithm

Your turn:

- Given the following graphs, present both graph and adjacency matrix for each graph and then trace-by-hand Floyd's algorithm to solve the all-pairs shortest path problem.



5.1.5.1.

	a	b	c	d
a	0	5	∞	∞
b	7	0	∞	2
c	∞	3	0	1
d	4	∞	1	0

5.1.4.2.

	a	b	c	d
a	0	1	∞	2
b	2	0	∞	2
c	∞	9	0	4
d	8	2	3	0

5.1.4.3.

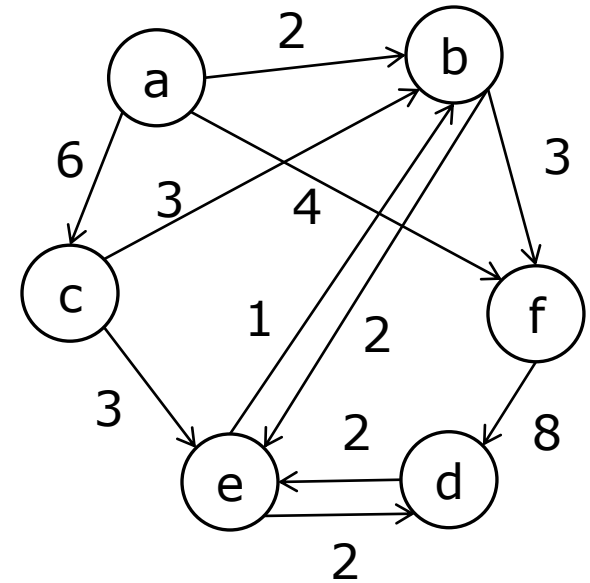
5.1.5. The all-pairs shortest path problem with Floyd's algorithm

Your turn:

- 5.1.5.4. Given a weighted graph:

 - What is its adjacency matrix?
 - What are its distance matrix and intermediate vertex matrix after the all-pairs shortest path problem is solved?
 - What are the shortest paths from **a** to **d** and from **c** to **f**?
- 5.1.5.5. Given an adjacency matrix:

 - What is its corresponding graph?
 - What are the shortest paths from **a** to **d** and from **d** to **b**?



	a	b	c	d
a	0	2	4	6
b	2	0	1	2
c	5	9	0	1
d	9	∞	2	0



5.2. Greedy algorithms

- ❑ Algorithms for optimization problems typically go through a sequence of steps, with a set of choices at each step. A greedy algorithm always makes the choice that *looks the best at the moment*.
- ❑ That is, it makes a locally optimal choice in the hope that this choice will lead to a globally optimal solution.
- ❑ Some examples of greedy algorithms for solving:
 - An activity-selection problem
 - The fractional knapsack problem
 - Huffman code problem
 - Finding minimum-spanning trees
 - Graph coloring



5.2. Greedy algorithms

- ❑ There are two ingredients that are exhibited by most problems that lend themselves to a greedy strategy: (1) the greedy choice property and (2) optimal substructure.
- ❑ Greedy choice property
 - The choice made by a greedy algorithm may *depend on choices so far, but it cannot depend on any future choices or on the solutions of the subproblems*. Thus, unlike dynamic programming, a greedy algorithm usually progresses in a *top-down* fashion, making one greedy choice after another, iteratively reducing each given problem instance to a smaller.
- ❑ Optimal substructure
 - The optimal solution for the problem contains within it optimal solutions to subproblems.



5.2. Greedy algorithms

Your turn:

□ 5.2.1. Compare the following strategies:

- Divide-and-conquer
- Decrease-and-conquer
- Dynamic programming
- Greedy

□ 5.2.2. Link each algorithm to its corresponding design strategy:

- | | |
|--|--------------------------|
| a. Algorithm for 0-1 knapsack problem | i. Divide-and-conquer |
| b. Algorithm for external sorting | ii. Decrease-and-conquer |
| c. Algorithm for constructing Huffman code | iii. Dynamic programming |
| d. Algorithm for finding topological sorting | iv. Greedy |



5.2.1. Activity-selection problem

- Suppose we have a set $S = \{1, 2, \dots, n\}$ of n activities that wish to use a *resource*, such as a lecture hall, which can be used by only one activity at a time.
- Each activity i has a *starting time* s_i and a *finishing time* f_i , such that $s_i \leq f_i$. If selected, activity i takes place during the half-open time interval $[s_i, f_i)$. Activities i and j are *compatible* if the intervals $[s_i, f_i)$ and $[s_j, f_j)$ do not overlap:

i and j are *compatible* if $s_i \geq f_j$ or $s_j \geq f_i$

- The activity-selection problem is to select a *maximum-size set* of mutually compatible activities.



5.2.1. Activity-selection problem

In the greedy algorithm for activity-selection problem, we assume that the input activities are *in ascending order of finishing times*: $f_1 \leq f_2 \leq \dots \leq f_n$.

```
procedure GREED-ACTIVITY-SELECTOR(s, f) ;    /* s is the
array keeping the starting times of the set of n activities and f is
the array keeping the finishing times of the set of n activities */
begin
  n := length[s];  A := {1};  j := 1;
  for i := 2 to n do
    if  $s_i \geq f_j$  then /* i is compatible with all activities in A */
      begin
        A := A  $\cup$  {i};
        j := i
      end
  end
```



5.2.1. Activity-selection problem

- ❑ The activity picked next by GREEDY-ACTIVITY-SELECTOR is always the one with *the earliest finishing time* that can be legally scheduled. The activity picked is thus “greedy choice” in the sense that *it leaves as many opportunities as possible for the remaining activities to be scheduled*.
- ❑ Greedy algorithms do not always produce optimal solutions. However, GREEDY-ACTIVITY-SELECTOR always finds an optimal solution to an instance of the activity-selection problem.



5.2.1. Activity-selection problem

Given a *sorted* list of activities, what activities are selected?

i	s_i	f_i
1	1	4
2	3	5
3	0	6
4	5	7
5	3	8
6	5	9
7	6	10
8	8	11
9	8	12
10	2	13
11	12	14





5.2.1. Activity-selection problem

Given a *sorted* list of activities, what activities are selected?

i	s_i	f_i	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
1	1	4															
2	3	5															
3	0	6															
4	5	7															
5	3	8															
6	5	9															
7	6	10															
8	8	11															
9	8	12															
10	2	13															
11	12	14															



5.2.1. Activity-selection problem

Given a *sorted* list of activities, what activities are selected?

i	s_i	f_i	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
1	1	4															
2	3	5															
3	0	6															
4	5	7															
5	3	8															
6	5	9															
7	6	10															
8	8	11															
9	8	12															
10	2	13															
11	12	14															

A = {1}



5.2.1. Activity-selection problem

Given a *sorted* list of activities, what activities are selected?

i	s_i	f_i	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
1	1	4															
2	3	5				---	---										
3	0	6	---	---	---	---	---	---									
4	5	7															
5	3	8															
6	5	9															
7	6	10															
8	8	11															
9	8	12															
10	2	13															
11	12	14															

A = {1}



5.2.1. Activity-selection problem

Given a *sorted* list of activities, what activities are selected?

i	s_i	f_i	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
1	1	4															
2	3	5				---	---										
3	0	6	---	---	---	---	---	---									
4	5	7															
5	3	8															
6	5	9															
7	6	10															
8	8	11															
9	8	12															
10	2	13															
11	12	14															

A = {1, 4}



5.2.1. Activity-selection problem

Given a *sorted* list of activities, what activities are selected?

i	s_i	f_i	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
1	1	4															
2	3	5				---	---										
3	0	6	---	---	---	---	---	---									
4	5	7															
5	3	8				---	---	---	---	---							
6	5	9						---	---	---	---						
7	6	10							---	---	---	---					
8	8	11															
9	8	12															
10	2	13															
11	12	14															

A = {1, 4}



5.2.1. Activity-selection problem

Given a *sorted* list of activities, what activities are selected?

i	s_i	f_i	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
1	1	4															
2	3	5				---	---										
3	0	6	---	---	---	---	---	---									
4	5	7															
5	3	8				---	---	---	---	---							
6	5	9						---	---	---	---						
7	6	10							---	---	---	---					
8	8	11															
9	8	12															
10	2	13															
11	12	14															

A = {1, 4, 8}



5.2.1. Activity-selection problem

Given a *sorted* list of activities, what activities are selected?

i	s_i	f_i	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
1	1	4															
2	3	5				---	---										
3	0	6	---	---	---	---	---	---									
4	5	7															
5	3	8				---	---	---	---	---							
6	5	9						---	---	---	---						
7	6	10							---	---	---	---					
8	8	11															
9	8	12										---	---	---	---		
10	2	13			---	---	---	---	---	---	---	---	---	---	---		
11	12	14															

$A = \{1, 4, 8\}$



5.2.1. Activity-selection problem

Given a *sorted* list of activities, what activities are selected?

i	s_i	f_i	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
1	1	4															
2	3	5				---	---										
3	0	6	---	---	---	---	---	---									
4	5	7															
5	3	8				---	---	---	---	---							
6	5	9						---	---	---	---						
7	6	10							---	---	---	---					
8	8	11															
9	8	12															
10	2	13			---	---	---	---	---	---	---	---	---	---	---		
11	12	14															

$A = \{1, 4, 8, 11\}$



5.2.1. Activity-selection problem

- ❑ Time complexity of the greedy algorithm for the activity-selection problem (not counting the cost of sorting *finishing times*)
 - Abstract operation: *comparison*
 - Best, Average, Worst cases: $O(n)$
- ❑ Time complexity of the greedy algorithm for the activity-selection problem (counting the cost of sorting *finishing times*)
 - Abstract operation: *comparison*
 - Best, Average, Worst cases: $O(n \log_2 n)$



5.2.1. Activity-selection problem

Your turn:

- Given the following lists of activities, from each list, what activities are selected for scheduling?

i	1	2	3	4	5	6	7	8	9	10	11	12
---	---	---	---	---	---	---	---	---	---	----	----	----

5.2.1.1. List 1

s_i	1	3	4	2	6	4	10	12	11			
f_i	3	4	6	8	9	13	14	15	16			

5.2.1.2. List 2

s_i	2	5	3	4	1	3	7	5	2	8	6	10
f_i	5	7	8	11	9	6	10	8	11	12	13	15



5.2.2. Fractional knapsack problem

- In the previous section with *dynamic programming*, the *0-1 knapsack problem* is posed as follows.

“A thief robbing a store finds it filled with N types of items of varying size and value (each item of the i -th item type is worth v_i dollars and weights w_i pounds), but has only a small knapsack of capacity M to use to carry the goods. The knapsack problem is to find the combination of items which the thief should choose for his knapsack in order to maximize the total value of all the items he takes.”

- This is called the 0-1 knapsack problem because each item must be either *taken or left behind*; the thief can *not take a fractional part* of an item.



5.2.2. Fractional knapsack problem

- ❑ In the fractional knapsack problem, the setup is the same, but the thief can take *fractions of items*, rather than having to make a binary (0-1) choice of each item.
- ❑ Both knapsack problems exhibit the *optimal substructure* property.
 - For the 0-1 problem, consider the most valuable load with the weights at most M pounds. If we remove item j from this load, the remaining load is the most valuable load weighing at most $M - w_j$ that the thief can take from the $n-1$ original items excluding j .
 - For the fractional problem, consider that if we remove a weight $w_j - w$ of one item j from the optimal load, the remaining load is the most valuable load weighing at most $M - (w_j - w)$ that the thief can take from the $n-1$ original items, excluding item j .
- ❑ We use *greedy algorithm* for the fractional knapsack and *dynamic programming* for the 1-0 knapsack.



5.2.2. Fractional knapsack problem

- To solve the fractional problem with the greedy strategy, we first compute *the value per pound* (v_i/w_i) for each item.
- The thief begins by taking as much as possible of the item with the greatest value per pound (v_i/w_i). If the supply of that item is exhausted and he still can carry more, he takes as much as possible of the item with the next greatest value per pound, and so forth until he cannot carry any more.



5.2.2. Fractional knapsack problem

```
procedure GREEDY_KNAPSACK( $V, W, M, X, n$ );  
var rc: real; i: integer;  
begin  
  for i:= 1 to n do  $X[i] := 0$ ;  
  rc := M ; // rc = remaining knapsack capacity  
  for i := 1 to n do  
    begin  
      if  $W[i] > rc$  then exit;  
       $X[i] := 1$ ; rc := rc –  $W[i]$   
    end;  
  if  $i \leq n$  then  $X[i] := rc/W[i]$   
end
```

V and W are the arrays that contain the values and weights of n objects ordered in such a way that $V_i/W_i \geq V_{i+1}/W_{i+1}$.

M is the knapsack capacity and X is a solution vector.



5.2.2. Fractional knapsack problem

- With the knapsack $M = 50$ and the following items:
 - $W[1] = 10, V[1] = 60, W[2] = 20, V[2] = 100, W[3] = 30, V[3] = 120, W[4] = 40, V[4] = 120$
- What is the combination of items that makes the total value of the knapsack the highest?

Item 1	Item 2	Item 3	Item 4
60	100	120	120

Knapsack
Max?



5.2.2. Fractional knapsack problem

- With the knapsack $M = 50$ and the following items:
 - $W[1] = 10, V[1] = 60, W[2] = 20, V[2] = 100, W[3] = 30, V[3] = 120, W[4] = 40, V[4] = 120$
- What is the combination of items that makes the total value of the knapsack the highest?

$$\frac{V[1]}{W[1]} = 6, \quad \frac{V[2]}{W[2]} = 5, \quad \frac{V[3]}{W[3]} = 4, \quad \frac{V[4]}{W[4]} = 3$$

→ Pick item 1, remaining capacity = $M - W[1] = 40$

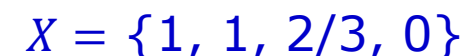
→ Pick item 2, remaining capacity = $40 - W[2] = 20$

→ Pick $\left(\frac{2}{3}\right) * \text{item 3}$, remaining capacity = $20 - \left(\frac{2}{3}\right) * W[3] = 0$

→ $X = \{1, 1, 2/3, 0\}$



- | Item 1 | Item 2 | Item 3 | Item 4 |
|--------|--------|--------|--------|
| | | | |
| | | | |
| | | | |
| | | | |
- 60 100 120 120





5.2.2. Fractional knapsack problem

- ❑ Time complexity of the greedy algorithm for solving the fractional knapsack problem (not counting the cost of sorting n objects)
 - Abstract operation: *comparison*
 - Best, Average, Worst cases: $O(n)$
- ❑ Time complexity of the greedy algorithm for solving the fractional knapsack problem (counting the cost of sorting n objects)
 - Abstract operation: *comparison*
 - Best, Average, Worst cases: $O(n \log_2 n)$



5.2.2. Fractional knapsack problem

Your turn:

- ❑ Given the knapsack and the following items, what is the combination of items that makes the total value of the knapsack the highest?
 - 5.2.2.1. $M = 20$, $W[A] = 18$, $V[A] = 25$, $W[B] = 15$, $V[B] = 24$, $W[C] = 10$, $V[C] = 15$
 - 5.2.2.2. $M = 10$, $W[A] = 5$, $V[A] = 9$, $W[B] = 4$, $V[B] = 8$, $W[C] = 2$, $V[C] = 3$, $W[D] = 3$, $V[D] = 5$
- ❑ 5.2.2.3. Design and analyze a greedy algorithm for the fractional knapsack problem where each item type has the limited number of items greater than or equal to 1.



5.2.3. Huffman codes

- ❑ We consider the problem of designing a binary character code wherein each character is represented by a unique binary string.
- ❑ Huffman codes are widely used for file compression.
- ❑ Huffman algorithm uses a table of the frequencies of occurrences of each character as a binary string.



5.2.3. Huffman codes

	a	b	c	d	e	f
Frequency	45	13	12	16	9	5
Fixed length codeword	000	001	010	011	100	101
Variable length codeword	0	101	100	111	1101	1100

- ❑ Suppose a 100,000-character data file needs to be stored compactly.
- ❑ If a *fixed length code* (3 bit) is used to represent 6 characters, 300,000 bits are needed to code the entire file of 100,000 characters.
- ❑ A *variable-length code* can do better than a fixed length code, by giving frequent characters short code-words and infrequent characters long code-words.
- ❑ This variable-length code requires:
 $(45 \cdot 1 + 13 \cdot 3 + 12 \cdot 3 + 16 \cdot 3 + 9 \cdot 4 + 5 \cdot 4) \cdot 100,000 / 100 = 224,000$ bits
to represent the file, saving approximately $\approx 25 \%$.



5.2.3. Huffman codes

□ Prefix-free code

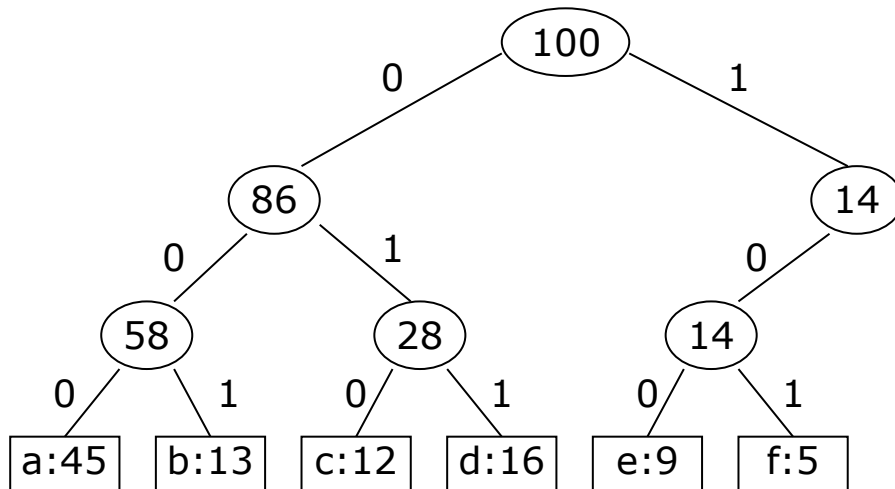
- We consider here only codes in which no codeword is also a prefix of some other codeword. Such codes are called *prefix-free-code* or *prefix-code*.
- It is possible to show that the optimal data compression achieved by a character code can always be achieved with a prefix code.
- Prefix codes are desirable because they simplify encoding and decoding.
- Encoding is simple: we just concatenate the code-words representing each character of the file.
- Decoding is simple with a prefix code. Since no codeword is a prefix of any other, the code word that begins an encoded file is unambiguous.



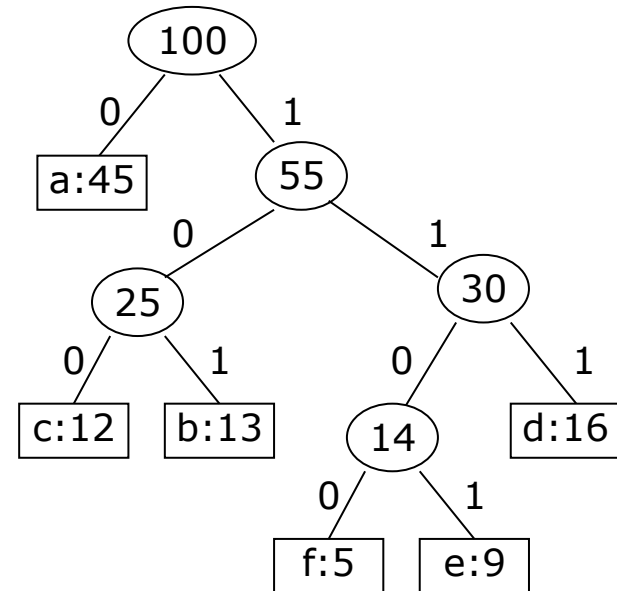
5.2.3. Huffman codes

- An optimal code for a file is always represented by a *full binary tree* in which every non-leaf node has two children.
- We interpret the binary codeword for a character as the path from the root to that character, where 0 means “*go to the left child*” and 1 means “*go to the right child*”.
- If C is the alphabet from which the characters are drawn, then the tree for an optimal prefix code has exactly $|C|$ leaves, one for each letter of the alphabet, and exactly $|C|-1$ internal nodes.

5.2.3. Huffman codes



(a). Fixed-length codes



(b). Variable-length codes

5.2.3. Huffman codes

- Given a tree T corresponding to a prefix code, it is a simple matter to compute the number of bits required to encode a file.
- For each character c in the alphabet C , let $f(c)$ denote the frequency of c in the file and $d_T(c)$ is the length of the codeword for character c . The number of bits required to encode a file is

$$B(T) = \sum_{c \in C} f(c) d_T(c)$$

which we define as the *cost* of the tree T .



5.2.3. Huffman codes

- Construct Huffman codes with the greedy strategy
 - Huffman invented a greedy algorithm that constructs an optimal prefix code called a *Huffman code*.
 - The algorithm builds the tree T corresponding to the optimal code in a bottom-up manner. It begins with a set of $|C|$ leaves and performs a sequence of $|C|-1$ “merging” operations to create the final tree.
 - A *priority queue* Q , keyed on f , is used to identify the two least frequency objects to merge together.
 - The result of merging the two objects is a new object whose frequency is the sum of the frequencies of the two objects that have been merged.

5.2.3. Huffman codes

□ The Huffman greedy algorithm

```
procedure HUFFMAN(C) ;  
begin  
  n := |C| ; Q := C ; // construct a min-heap  
  for i := 1 to n - 1 do  
    begin  
      z := ALLOCATE-NODE( ) ;  
      left[z] := EXTRACT-MIN(Q) ;  
      right[z] := EXTRACT-MIN(Q) ;  
      f[z] := f[left[z]] + f[right[z]] ;  
      INSERT(Q, z) ;  
    end  
  end
```

5.2.3. Huffman codes

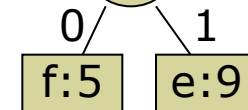
- Given the following characters and their frequencies in textual files.
 - Construct a Huffman code for these characters.
 - What is the average code length?
 - Compared to a fixed-length code, how much does this Huffman code save when compressing a file of 200,000 characters?

Character	a	b	c	d	e	f
Frequency	45	13	12	16	9	5

5.2.3. Huffman codes

(i) f:5 e:9 c:12 b:13 d:16 a:45

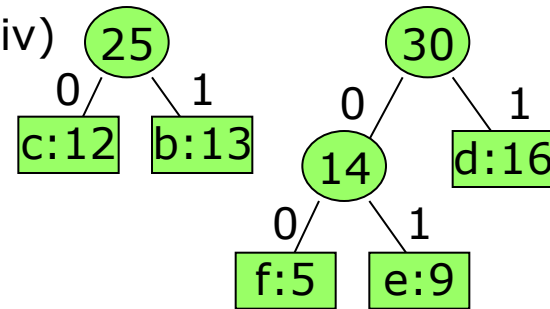
(ii) c:12 b:13 14 d:16 a:45



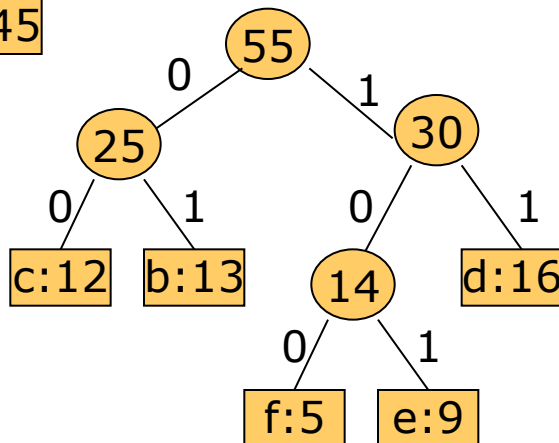
(iii) 14 d:16 25 a:45



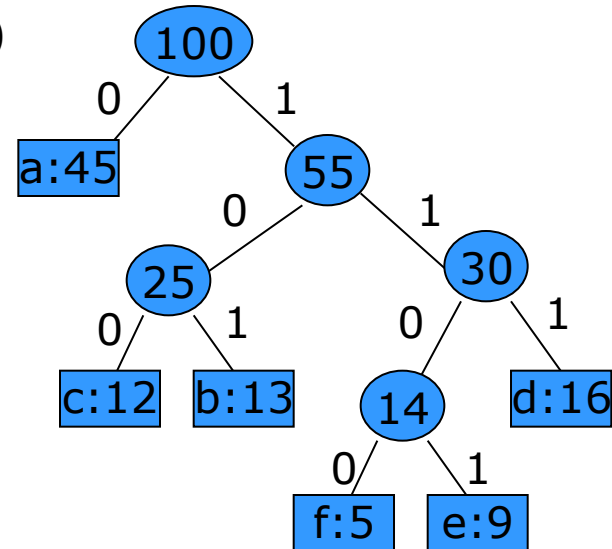
(iv) 25 30 a:45



(v) a:45



(vi)





5.2.3. Huffman codes

- Given the following characters and their frequencies in textual files.

Character	a	b	c	d	e	f
Frequency	45	13	12	16	9	5

Huffman code:

a = 0

b = 101

c = 100

d = 111

e = 1101

f = 1100

Averaged code length = $(1*45 + 3*13 + 3*12 + 3*16 + 4*9 + 4*5)/100 = 2.24$ bits/character

Fixed-length code:

code length = $\lceil \log_2 6 \rceil = 3$ bits/character

Requires: $200,000 * 3 = 600,000$ bits

Huffman code:

Requires: $200,000 * 2.24 = 448,000$ bits

Saved: $(3 - 2.24)/3 = 25.33\%$



5.2.3. Huffman codes

- Time complexity of the Huffman greedy algorithm to construct Huffman codes on a set of n characters
 - Abstract operation: *comparison*
 - Best, Average, Worst cases: $O(n \log_2 n)$
 - Given a set C of n characters, building Q as a min-heap can be done with time $O(n)$.
 - The *for* loop is executed exactly $n-1$ times, and since each heap operation requires $O(\log_2 n)$, this loop contributes $O(n \log_2 n)$ to the running time.



5.2.3. Huffman codes

Your turn:

- ❑ 5.2.3.1. Given the following characters and their frequencies in textual files.
 - Construct a Huffman code for these characters.
 - What is the average code length?
 - Compared to a fixed-length code, how much does this Huffman code save when compressing a file of 300,000 characters?

Character	a	b	c	d	e	f	g
Frequency	23	18	12	15	20	5	7

5.2.3. Huffman codes

Your turn:

- ❑ 5.2.3.2. Given the following characters and their frequencies in textual files.
 - Construct a Huffman code for these characters
 - Encode the text ABACAB_D using the Huffman code achieved above.
 - Decode the text whose encoding is 10011110111001010 in the Huffman code achieved above.

Character	A	B	C	D	_
Frequency	40	10	20	15	15

5.2.3. Huffman codes

Your turn:

- ❑ 5.2.3.3. Given the following characters and their frequencies in textual files.
 - Construct a Huffman code for these characters.
 - What is the average code length?
 - Compared to a fixed-length code, how much does this Huffman code save when compressing a file of 500,000 characters?

Character	A	B	C	D	E
Frequency	20	7	10	4	18

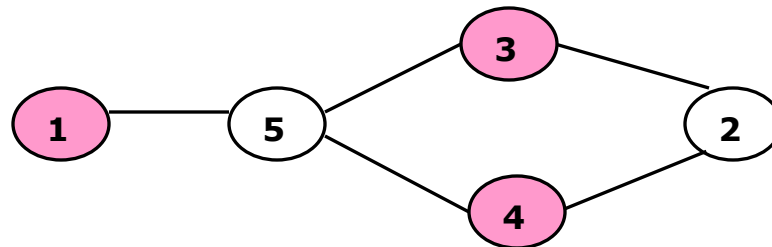


5.2.4. Graph coloring

- Given an undirected graph, *coloring* the graph is an assignment of a color to each vertex of the graph so that no two vertices connected by an edge can have the same color. We wish to find a coloring with the *minimum number of colors*.
- This is an optimization problem.
- One reasonable strategy for graph coloring is using a greedy algorithm.
- The idea: *Initially, we try to color as many vertices as possible with the first color, then as many uncolored vertices as possible with the second color and so on.*
- Note: Greedy algorithm *cannot* yield the optimal solution for this problem.

5.2.4. Graph coloring

- ❑ To color vertices with a new color, we do:
 - Select some uncolored vertex and color it with a new color.
 - Scan the list of uncolored vertices. For each uncolored vertex, determine whether it has an edge to any vertex already colored with the new color. If there is no such an edge, color the present vertex with the new color.
- ❑ In the following figure, we color vertex 1 with pink color and then color vertices 3 and 4 with the same pink color.





5.2.4. Graph coloring

- Procedure SAME_COLOR determines a set of vertices (called *newclr*), all of which can be colored with a new color. This procedure is called repeatedly until all vertices are colored.

```
procedure SAME_COLOR(G, newclr);  
  /* SAME_COLOR assigns to newclr a set of vertices of G that  
  may be given the same color */  
  begin  
    newclr :=  $\emptyset$ ;  
    for each uncolored vertex v of G do  
      if v is not adjacent to any vertex in newclr  
      then  
        mark v colored and add v to newclr.  
  end;
```



5.2.4. Graph coloring

```
procedure G_COLORING(G);  
  procedure SAME_COLOR(G, newclr);  
    /* SAME_COLOR assigns to newclr a set of  
       vertices of G that may be given the same color ;  
       a: adjacency matrix for graph G */  
  begin  
    newclr :=  $\emptyset$ ;  
    for each uncolored vertex v of G do  
      begin  
        found := false;  
        for each vertex w  $\in$  newclr do  
          if a[v,w] = 1 /* there is an edge between v and w in G */  
            then  
              found := true;  
          if not found then  
            mark v colored and add v to newclr  
        end  
      end  
    end;
```



5.2.4. Graph coloring

```
for each vertex in G do mark uncolored;  
while there is any vertex marked uncolored do  
begin  
    SAME_COLOR(G, newclr);  
    print newclr  
end;
```

- Theorem: If $\chi(G)$ is the minimum number of colors to color graph G and ΔG is the largest degree in G then $\chi(G) \leq \Delta G + 1$.

Complexity of the greedy algorithm for graph coloring

- Assume that the graph is represented by an adjacency matrix.
- In procedure SAME_COLOR, each cell in the adjacency matrix is examined when we color the uncolored vertices with a new color.
- Complexity of procedure SAME_COLOR: $O(n^2)$ where n is the number of vertices in G .
- If m is the number of colors used to color the graph then procedure SAME_COLOR is called m times at all. Therefore, the complexity of the whole algorithm is $m \cdot O(n^2)$. Since m is often a small number, we can say: the algorithm has a quadratic complexity.



5.2.4. Graph coloring

□ Applications

■ Exam timetabling

- Each exam is represented by a vertex in the graph.
- Exam timetabling is assigning time periods to exams. Time periods are the colors used to color the vertices of the graph.
- An edge connects two vertices if there exists at least one student who takes both the exams, therefore we are not allowed to assign those two exams which are represented by the two vertices to the same time period.

■ Frequency assignment problem in wireless broadcasting or mobile telephone



5.2.4. Graph coloring

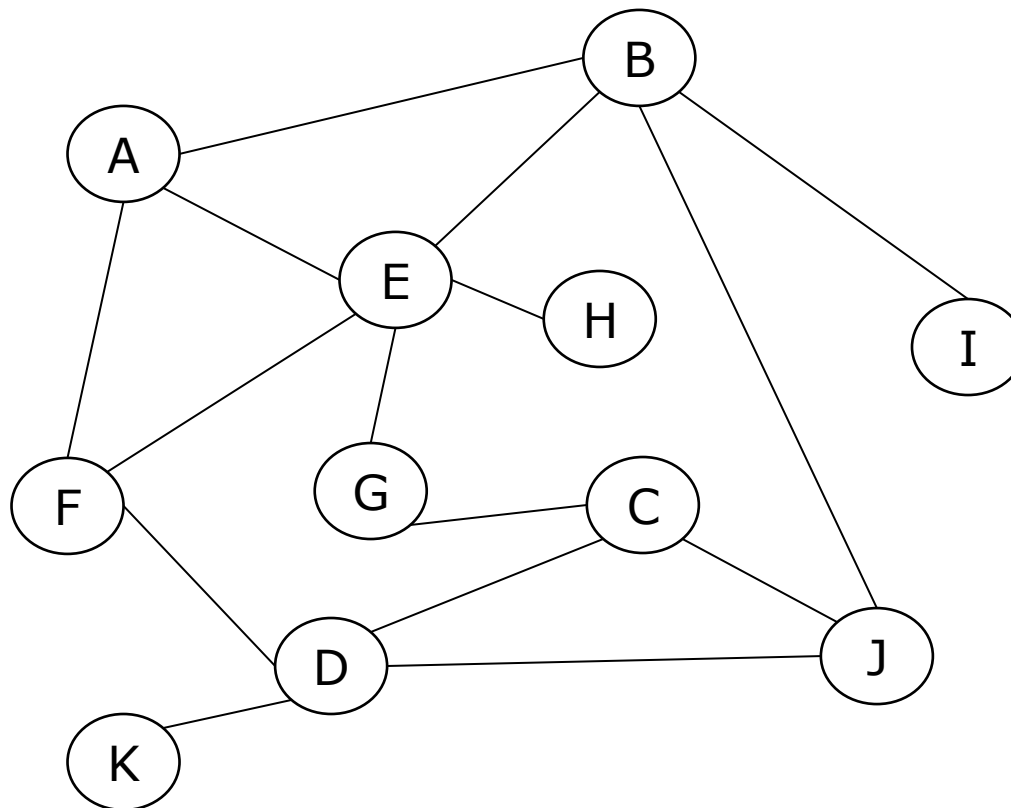
Heuristics for graph coloring

"The vertex with the largest degree will be examined to color first". (Welsh and Powell)

- *Degree of a vertex*: the number of edges connected to this vertex.
- Reason: The vertices with more connected edges will be more difficult to be colored if we wait until all their adjacent vertices are colored.
- Algorithm
 - 1. Arrange the vertices in descending order of degrees.
 - 2. Color a vertex with the maximum degree with color 1.
 - 3. Choose an uncolored vertex with the maximum degree. If there is another vertex with the same maximum degree, choose either of them.
 - 4. Color the chosen vertex with the least possible (lowest numbered) color.
 - 5. If all vertices are colored, stop. Otherwise, return to 3.

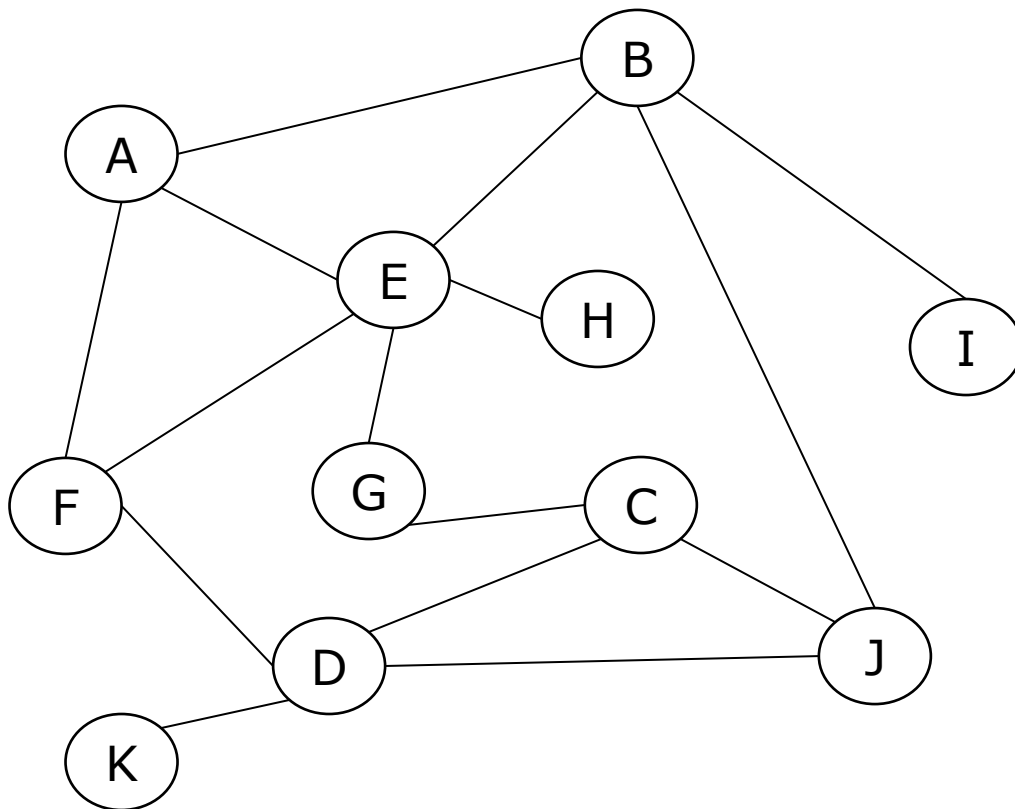
5.2.4. Graph coloring

- Given a graph as follows. Conduct graph coloring with Welsh and Powell's heuristic. How many colors have been used?



5.2.4. Graph coloring

- Given a graph as follows. Conduct graph coloring with Welsh and Powell's heuristic. How many colors have been used?



sort

E: 5
B: 4
D: 4
A: 3
C: 3
F: 3
J: 3
G: 2
H: 1
I: 1
K: 1

coloring

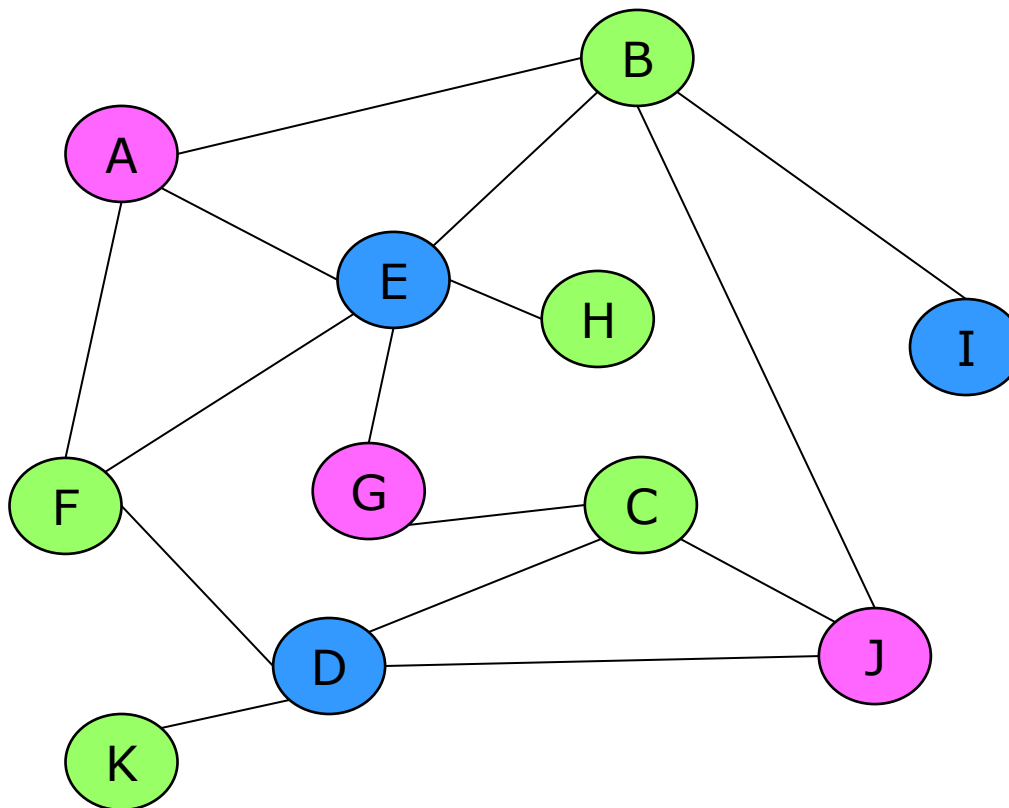
Color E, D, I with
blue

Color B, C, F, H, K
with green

Color A, J, G with
pink

5.2.4. Graph coloring

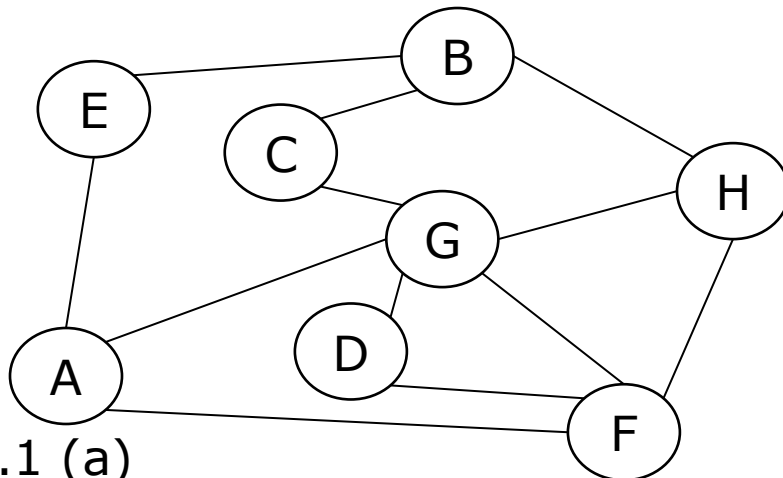
- Given a graph as follows. Conduct graph coloring with Welsh and Powell's heuristic. How many colors have been used? **3**



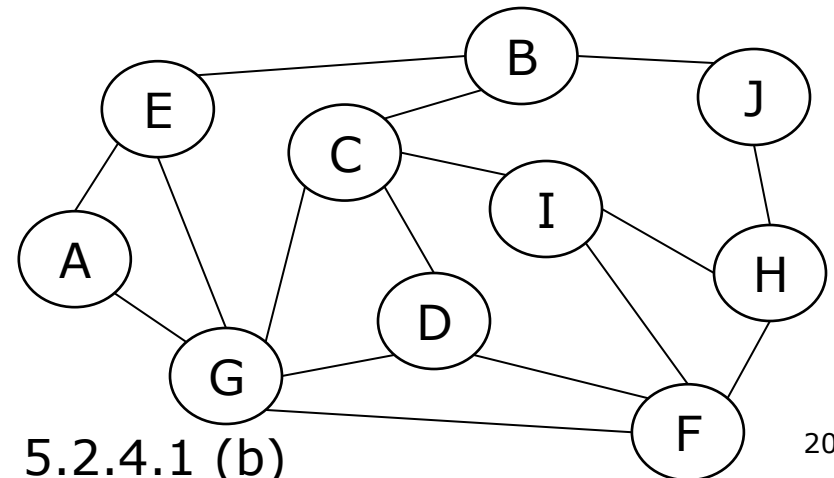
5.2.4 Graph coloring

Your turn:

- 5.2.4.1. Given a graph as follows. Color this graph using a color set of {green, red, yellow} in such a way that no two vertices connected by an edge can have the same color. If using Welsh and Powell's heuristic, do you obtain the same result?



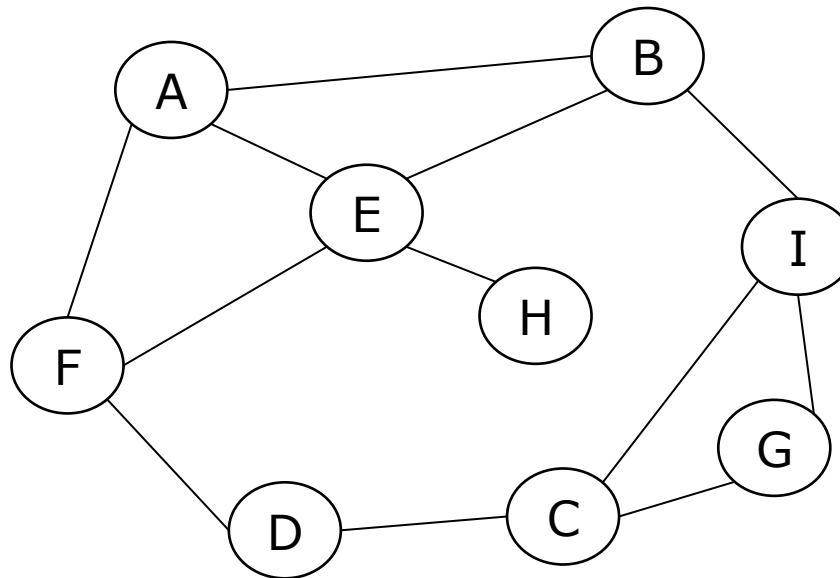
5.2.4.1 (a)



5.2.4.1 (b)

5.2.4 Graph coloring

- 5.2.4.2. Given a graph as follows. Color this graph in such a way that no two vertices connected by an edge can have the same color. If using Welsh and Powell's heuristic, do you obtain the same result?

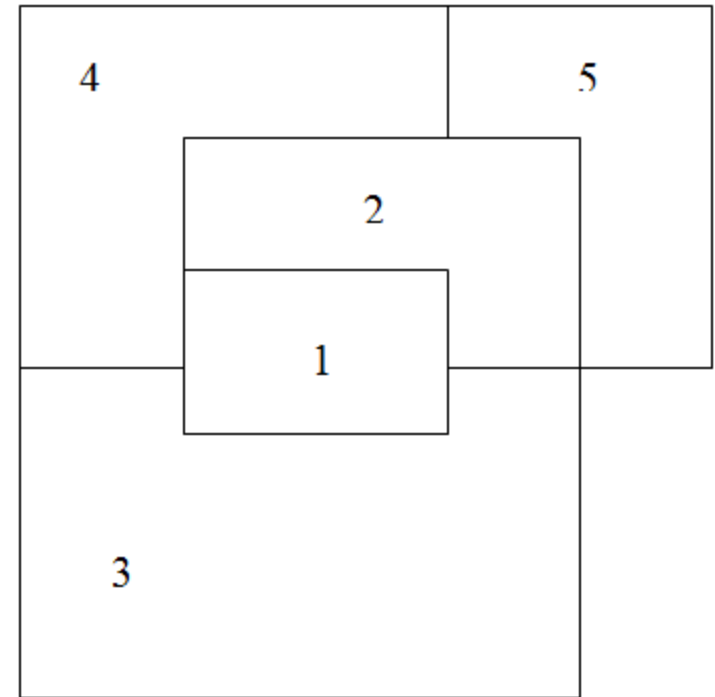




5.2.4 Graph coloring

Your turn:

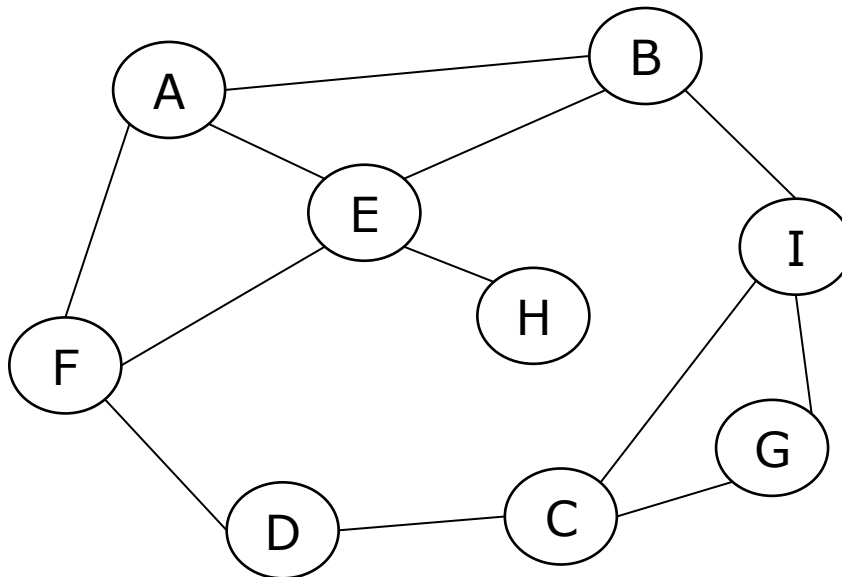
- 5.2.4.3. Given a map as follows. Color this map in such a way that no two adjacent regions can have the same color. If using Welsh and Powell's heuristic, do you obtain the same result?



5.2.4 Graph coloring

Your turn:

- 5.2.4.4. Given a graph as follows. Color the edges of this graph in such a way that no two edges sharing the same vertex can have the same color. Is the number of colors you have used the minimum one? Why?



Algorithms Analysis and Design (C03031)

□ 5.1. Dynamic Programming

- 5.1.1. Matrix-chain multiplication
- 5.1.2. Longest common subsequence
- 5.1.3. (0-1) Knapsack problem
- 5.1.4. Transitive closure with Warshall's algorithm
- 5.1.5. The all-pairs shortest path problem with Floyd's algorithm



□ 5.2. Greedy Algorithms

- 5.2.1. Activity-selection problem
- 5.2.2. Fractional knapsack problem
- 5.2.3. Huffman codes
- 5.2.4. Graph coloring